

Plan de Empresa

Irene González García

Inmaculada Hortelano

Tutor: Aníbal Martín

Memoria del Proyecto Técnico "Inspire Beauty"

Curso 2025/26

**Grado Superior
DAW online**



Contenido

1. Agradecimientos/dedicatoria
3. Resumen
4. Abstract (traducción del resumen a inglés)
5. Introducción
6. Alcance funcional del sistema
7. Diseño técnico
8. Tecnología utilizada
9. Metodología y planificación
10. Manual de instalación
11. Manual de usuario
12. Manual del Personal de la Clínica (Modo Admin)
13. Plan de Pruebas y Validación (Testing)
14. Conclusiones
15. Bibliografía y Webgrafía



1. Agradecimientos y Dedicatoria

A mi compañera de proyecto, Inmaculada Hortelano, por su paciencia, su impecable trabajo en el Backend y por ser el apoyo constante durante estos meses de desarrollo. Gracias por hacer que la integración de "Inspire Beauty" fuera un proceso de aprendizaje mutuo y éxito compartido.

A mis profesores, especialmente a mi tutor de proyecto, por guiarnos con sus conocimientos técnicos y por motivarnos a alcanzar un nivel profesional en nuestras aplicaciones.

A mi familia, por su apoyo incondicional y por creer en mis capacidades durante todo este ciclo formativo. Sin su ánimo, este trabajo no habría sido posible.

- Irene González

A mi compañera, Irene González – de proyecto y de curso – por ser esa fuente borboteante de ideas que ha empujado el nivel de este proyecto hasta lo más excelso, cuestionando nuestras propias capacidades e inexperiencia. Por su brillante diseño del Frontend que con tanta destreza ha creado, haciendo de este proyecto un proceso de mejoras continuas y lecciones aprendidas.

A los profesores – sobre todo Alberto, que su paso fugaz nos enseñó lo que no aparece en el temario: cómo aprender – por su incansable labor y por soportarnos – a unos más que a otros, me temo –. No es nada fácil llevar auestas la exigencia de un Grado Superior de DAW online.

A mi pareja, Marcos Sánchez, por su apoyo más que incondicional, por aguantar mi frustración (y mis traspasos) con los famosos "saltos de fe" y sobre todo por no rendirse conmigo cuando yo ya me había rendido con Java.

Y, por último, a mi familia – la de sangre y de no-sangre – por todo el apoyo recibido, los ánimos, y por encima de cualquier otra cosa, aguantar estoicamente este otro giro drástico a mi carrera profesional viniendo de una carrera de humanidades puras. *Alea iacta est et nobis arridet.*

- Inmaculada Hortelano

2. Resumen

El presente proyecto técnico aborda el diseño, desarrollo e implantación de "Inspire Beauty", una aplicación web integral orientada a la modernización y digitalización de los procesos de gestión en un centro de estética y bienestar. Tradicionalmente, la administración de citas, el control de servicios y la captación de clientes en este sector se han fundamentado en metodologías analógicas que limitan la escalabilidad del negocio. Para dar respuesta a esta problemática, se ha construido una plataforma dual que satisface tanto las necesidades transaccionales del cliente final como los requerimientos operativos del personal administrativo.

Desde el punto de vista tecnológico, el sistema se asienta sobre una arquitectura cliente-servidor basada en el patrón de diseño MVC (Modelo-Vista-Controlador). La capa de presentación (Frontend) se ha desarrollado prescindiendo deliberadamente de frameworks de terceros, empleando un maquetado purista con HTML5 semántico, CSS3 avanzado (basado en esquemas de Grid y Flexbox) y JavaScript nativo. Esta decisión garantiza un rendimiento óptimo y un diseño Mobile First altamente responsivo. Simultáneamente, la lógica de negocio y servidor (Backend) ha sido programada íntegramente en PHP, asegurando la persistencia y portabilidad de los datos mediante el motor de base de datos SQLite, gestionado a través de la interfaz de acceso seguro PDO (PHP Data Objects).

El resultado final es una solución de software robusta, segura y escalable que optimiza los flujos de trabajo del centro. La herramienta permite el procesamiento dinámico de reservas, la automatización en el control de la agenda del personal, y funciona como un potente escaparate digital que refuerza y moderniza la identidad de marca del negocio.

3. Abstract

This project focuses on the development of "Inspire Beauty", a comprehensive web application for managing appointments and services in an aesthetic center. The system provides customers with a visually appealing interface to browse facial and body treatment catalogs and contact the center through an optimized web form. Technically, the Frontend has been built using semantic HTML5, advanced CSS3 (Flexbox and Grid), and JavaScript, ensuring a fully responsive design. The application addresses the challenges of manual appointment scheduling, offering an efficient digital solution that enhances communication between the business and its clients.

4. Introducción

El sector de los servicios estéticos y de bienestar ha experimentado un crecimiento sostenido durante los últimos años, impulsado por una mayor concienciación sobre el cuidado personal. Sin embargo, una gran proporción de los centros de estética tradicionales continúa operando bajo modelos de gestión obsoletos, fundamentados en agendas físicas y canales de comunicación ineficientes, como la reserva telefónica exclusiva. Este déficit tecnológico no solo provoca cuellos de botella administrativos y errores humanos en el solapamiento de horarios, sino que también penaliza severamente la visibilidad del negocio frente a nuevos nichos de mercado digitalizados.

El presente Trabajo de Fin de Ciclo (TFC), desarrollado de manera colaborativa, nace con la vocación de resolver esta carencia empresarial mediante la creación de "Inspire Beauty": un ecosistema web centralizado y a medida. La motivación principal de este proyecto reside en demostrar cómo una ingeniería de software aplicada con rigor y buenas prácticas puede transformar por completo la operatividad de una PYME.

El objetivo general del proyecto es dotar al negocio de una plataforma unificada que resuelva la organización de citas y la gestión de venta de tratamientos. Para alcanzar este objetivo, el equipo de desarrollo ha trazado una estrategia que aúna dos pilares fundamentales: Por un lado, la capa de presentación (Frontend) ha sido diseñada como un escaparate digital profesional (Home Page y Catálogos), permitiendo captar nuevos prospectos y garantizando una experiencia de usuario (UX/UI) adaptativa, para que cualquier cliente pueda interactuar con el centro desde su ordenador o dispositivo móvil. Por otro lado, soportando esta interfaz, se ha desarrollado una sólida lógica de servidor (Backend). Se ha apostado por la implementación del paradigma arquitectónico Modelo-Vista-Controlador (MVC) en PHP, diseñando desde cero una base de datos relacional para el control transaccional de usuarios y citas, logrando así un entorno securizado y eficiente.

En los sucesivos apartados de esta memoria, se desgranarán las decisiones técnicas adoptadas en conjunto, la arquitectura de la información, el ecosistema de tecnologías implicadas y los flujos de validación que conforman la infraestructura de "Inspire Beauty", demostrando la viabilidad técnica y profesional del software entregado.

5. Alcance Funcional del Sistema

El proyecto "Inspire Beauty" se ha diseñado como una solución integral para la digitalización de centros de estética, articulándose en dos grandes bloques funcionales que interactúan bidireccionalmente:

5.1. Presencia Digital y Área Pública (Perfil Cliente)

- **Home Page Informativa:** Interfaz diseñada con CSS nativo que expone la propuesta de valor del centro, los servicios estrella y las promociones vigentes de forma visual e intuitiva.
- **Catálogo de Servicios:** Páginas específicas y detalladas para tratamientos faciales y corporales, mostrando descripciones, beneficios, duraciones estimadas y precios.
- **Sistema de Carrito de Compras:** Módulo que permite a los usuarios seleccionar múltiples tratamientos, visualizar el importe total y simular el proceso de pago y finalización de compra.
- **Canal de Contacto:** Formulario web optimizado con validaciones del lado del cliente y servidor para canalizar las dudas y consultas de clientes potenciales.
- **Motor de Reservas de Cita Online:** Sistema interactivo que permite al cliente seleccionar un tratamiento y proponer una fecha para la cita. La petición queda registrada en el sistema en estado "Pendiente de confirmación".

5.2. Gestión Administrativa y Área Privada (Perfil Centro)

- **Control de Acceso (Login):** Interfaz de autenticación securizada, exclusiva para las administradoras del centro.
- **Dashboard de Administración:** Panel de control centralizado desde donde se supervisan todas las operaciones del negocio en tiempo real.
- **Gestión de Agenda Interactiva:** Módulo que permite al personal confirmar, cancelar o posponer las solicitudes de cita de los clientes. El sistema automatiza lógicas de negocio críticas, como la asignación obligatoria de márgenes de limpieza (ej. 15 minutos) entre citas, evitando solapamientos mediante validaciones algorítmicas de rangos temporales.
- **Administración de Catálogo y Promociones:** Capacidad para modificar los precios de la clínica, activar o desactivar la visibilidad de ciertos tratamientos en la parte pública, y asignar precios promocionales. El sistema implementa validaciones estrictas para asegurar que el precio promocional configurado sea siempre inferior al precio base.
- **Bandeja de Mensajes y Pedidos:** Visor administrativo del historial de transacciones procesadas por los clientes desde la tienda y un buzón unificado de los formularios de contacto recibidos.

Ambos bloques comparten el requisito transversal de contar con un diseño adaptativo (*Responsive Design*), garantizando una experiencia de usuario (UX) fluida y homogénea independientemente del dispositivo (móvil, tablet o PC) bajo la estricta directriz metodológica *Mobile First*.

6. Diseño técnico

6.1. Diagrama de Clases y Arquitectura de Software

Para ilustrar la estructura lógica del servidor y las relaciones entre los distintos componentes del sistema, se ha diseñado el correspondiente Diagrama de Clases (UML). El diseño refleja fielmente el paradigma de Programación Orientada a Objetos (POO) y la arquitectura Modelo-Vista-Controlador (MVC) implementada en el Backend.

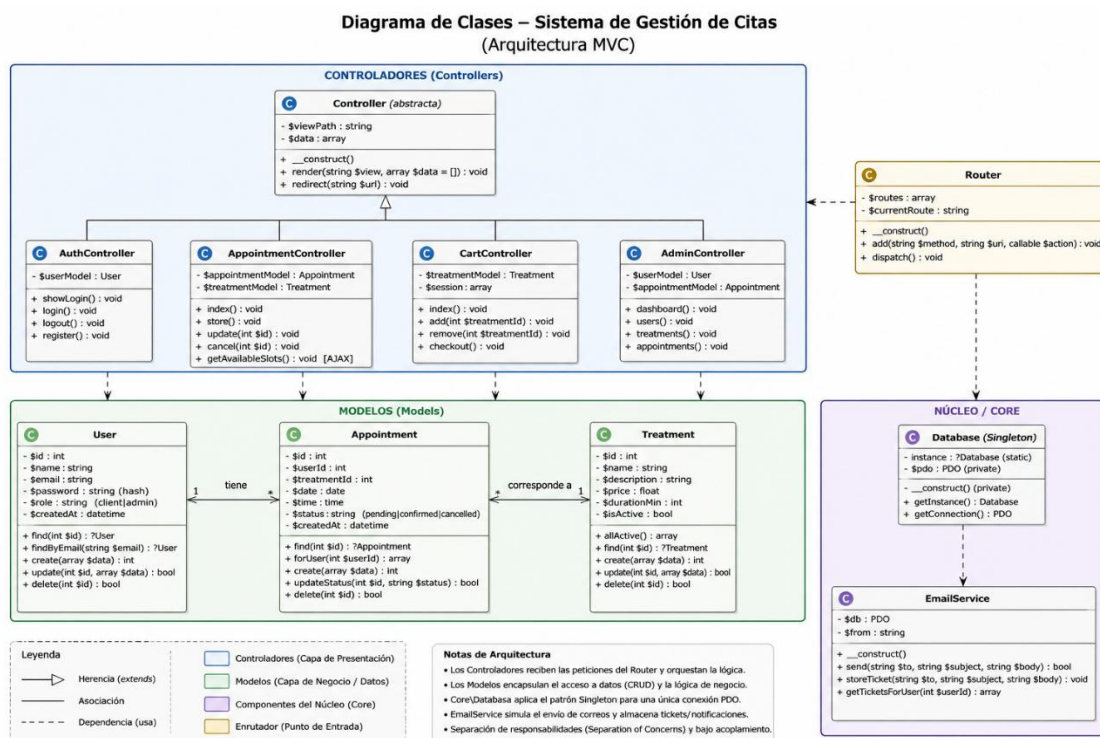


Ilustración 1: Diagrama de clases

Como se puede observar en el diagrama, la arquitectura se sustenta sobre las siguientes jerarquías y dependencias:

1. Capa de Controladores (Controllers): El sistema orbita en torno a una clase base Controller, la cual abstrae la lógica común de renderizado de vistas. De ella heredan todos los controladores específicos del dominio: AuthController (manejo de sesiones), AppointmentController (gestión de citas y AJAX), CartController (flujo de tienda) y AdminController (gestión avanzada del personal). Estos controladores actúan como orquestadores, recibiendo las peticiones del enrutador central y delegando la carga de trabajo a los modelos pertinentes.

2. Capa de Modelos (Models): Representan las entidades de negocio y encapsulan toda la lógica de interacción con la base de datos (CRUD). Las clases principales son User (clientes y administradores), Treatment (catálogo de servicios) y Appointment (agenda). Los modelos no interactúan directamente con las vistas, asegurando una separación de intereses (Separation of Concerns) estricta.

3. Componentes del Núcleo (Core): Para dar soporte transversal a la aplicación, se han diseñado clases de utilidad bajo el espacio de nombres Core:

La clase Database aplica el patrón de diseño Singleton, garantizando que exista una única instancia de la conexión PDO abierta durante el ciclo de vida de la petición HTTP, optimizando así el consumo de memoria.

La clase EmailService actúa como un microservicio interno, encapsulando la lógica de generación y almacenamiento temporal de notificaciones y tickets electrónicos (Simulador de correos).

Esta estructura de clases garantiza un código altamente cohesivo, débilmente acoplado, y preparado para futuras escalabilidades o integraciones de nuevos módulos.

6.2 Esquema de bases de datos

El repositorio de datos se sustenta en una base de datos relacional SQLite normalizada. Las tablas principales del esquema son:

- **users:** Entidad encargada de registrar tanto a los clientes como a los perfiles administrativos. Emplea el campo role para la segregación de privilegios (client o admin) y almacena las credenciales de acceso de forma segura (Hash Bcrypt).
- **treatments:** Catálogo central de los servicios ofertados por la clínica. Documenta el nombre, categoría, duración en minutos, márgenes de limpieza obligatorios, precio base y precio de promoción. Incluye un indicador booleano (is_active) para controlar su visibilidad pública.
- **appointments:** Entidad transaccional que enlaza a un usuario (cliente) con un tratamiento. Registra atributos críticos como la fecha de la cita, hora de inicio, hora de finalización (calculada matemáticamente en el backend) y el estado de la reserva.
- **contact_messages:** Registro histórico de todas las consultas enviadas mediante el formulario público de contacto.
- **purchases y purchase_items:** Tablas relacionadas (relación 1:N) que asientan las compras finalizadas desde el carrito, vinculando la cesta de tratamientos a un identificador de transacción y cliente.

6.3. Diseño de la Interfaz de Usuario (Frontend)

El desarrollo de la interfaz de usuario (UI) y la experiencia de usuario (UX) del proyecto "Inspire Beauty" ha sido liderado íntegramente por Irene González. El enfoque principal ha consistido en construir una plataforma estética, intuitiva y altamente responsiva, alineada con los valores de exclusividad y bienestar que transmite la marca.

Para la maquetación del proyecto, se ha optado por prescindir de frameworks pesados (como Bootstrap o Tailwind CSS), apostando en su lugar por el uso de **HTML5 Semántico y CSS3 puro (Vanilla CSS)**. Esta decisión técnica garantiza un control absoluto sobre cada píxel de la interfaz, reduce drásticamente los tiempos de carga al evitar la descarga de librerías innecesarias, y demuestra un dominio profundo de las hojas de estilo en cascada.

A continuación, se detallan los pilares fundamentales sobre los que se sustenta el diseño Frontend:

6.3.1. Identidad Visual y Tipografía

Para garantizar la máxima coherencia visual, escalabilidad y facilidad de mantenimiento a lo largo de toda la aplicación, se ha diseñado una arquitectura CSS estructurada apoyada en el uso de **Variables CSS nativas (*Custom Properties*)**.

En lugar de codificar los valores de color de forma rígida en los distintos selectores, se ha definido un diccionario de diseño en la raíz (:root) dentro del archivo principal estilos.css. Esta práctica centraliza el diseño y permite gestionar de forma unificada la paleta cromática de la marca, estructurándose en los siguientes pilares:

1. Paleta Premium e Identidad Visual: Los colores escogidos buscan transmitir relajación, profesionalidad y la elegancia propia de un centro de estética de alto nivel, basándose en tonos neutros y empolvados:

- **Tono Pastel (rgb(247, 230, 230)):** Un fondo cálido que envuelve al usuario y reduce la fatiga visual.
- **Gris Carbón (#4a4a4a):** Sustituye al negro puro en los textos principales para suavizar el contraste en pantalla, logrando una lectura mucho más agradable.
- **Gris Muted:** Utilizado para restar peso visual a elementos secundarios, estableciendo una correcta jerarquía de lectura.

2. Interactividad y Estados (:hover): Se ha prestado especial atención a la experiencia de usuario (UX) definiendo contrastes invertidos en los estados :hover. Por ejemplo, utilizando el color corporativo beige (#efeee5), se asegura que todos los botones y enlaces reaccionen de manera uniforme, proporcionando un *feedback* visual claro al interactuar con ellos.

Modularidad en las Vistas: Aunque el archivo estilos.css actúa como núcleo inyectado en el *Layout* general, el proyecto hace uso de hojas de estilo complementarias cargadas dinámicamente según la vista lo requiera. Gracias a la centralización de las variables globales, estas hojas secundarias pueden nutrirse de la misma paleta. Si en un futuro la clínica decidiera someterse a un *rebranding*, bastaría con modificar una única línea de código en el :root para que toda la plataforma se actualizase instantáneamente.

```
# estilos.css
TrabajaFinalP > frontend > css > # estilos.css > section2 h3
1 @import url("https://fonts.googleapis.com/css2?family=Lato:ital,wght@0,100;0,300;0,400;0,700;0,900;1,100;1,300;1,400;1,700;1,900&family=Playfair+Display:ital,wght@0,400..
2 /*Estilos Universales*/
3 * {
4   margin: 0;
5   padding: 0;
6   box-sizing: border-box;
7 }
8
9 h1,
10 h2,
11 h3 {
12   font-family: "Playfair Display", serif;
13   font-weight: 700; /* Esto le da el grosor elegante */
14   color: #4a4a4a; /* Un gris oscuro queda mas suave que el negro puro */
15 }
16
17 img {
18   max-width: 100%;
19 }
20
21 a {
22   text-decoration: none;
23   color: #000000;
24 }
25
26 .row {
27   display: flex;
28   flex-wrap: wrap;
29 }
30
31 body {
32   font-family: "Lato", sans-serif;
33 }
34
35 [class*="col-"] {
36   width: 100%;
```

Ilustración 2: Declaración de variables de color y fuentes tipográficas en estilos.css.

La paleta cromática se basa en tonos neutros y empolvados (como el rgb(247, 230, 230) presente en el CSS), combinados con grises oscuros suaves para el texto, evitando el negro puro para reducir la fatiga visual del usuario.

Para la concepción visual se desarrollaron wireframes de baja fidelidad que permitieron estructurar los contenidos antes de la codificación. A continuación, se muestra la evolución desde el boceto inicial hasta la implementación final en el navegador.

6.3.2. Identidad Tipográfica

La selección tipográfica juega un papel crucial en el diseño Frontend, especialmente cuando el objetivo es transmitir los valores de un centro de estética premium. Para lograr la estética deseada y, al mismo tiempo, asegurar una excelente accesibilidad y legibilidad, se han implementado dos familias tipográficas complementarias importadas directamente mediante Google Fonts.

Esta combinación establece un claro contraste jerárquico visual:

- **Playfair Display (Serif):** Aplicada de forma exclusiva en los encabezados principales de la aplicación (<h1>, <h2>, <h3>). Su diseño, caracterizado por el contraste pronunciado entre trazos gruesos y finos y su serifa clásica, aporta la sofisticación, el clasicismo y la elegancia inherentes a la identidad corporativa de la marca "Inspire Beauty". Funciona como un fuerte anclaje visual que capta la atención del usuario en los títulos de secciones y tratamientos.
- **Lato (Sans-Serif):** Utilizada extensivamente para los cuerpos de texto, descripciones largas, menús de navegación y elementos interactivos (como botones de llamada a la acción). Su diseño de palo seco (*sans-serif*), limpio, moderno y con proporciones equilibradas, garantiza una claridad y legibilidad óptimas en dispositivos digitales, evitando la fatiga visual del usuario durante procesos como la lectura del catálogo o la formalización de reservas.

La correcta combinación de ambas familias permite que la interfaz respire y guíe al usuario de forma intuitiva, diferenciando claramente qué es información destacada (títulos) y qué es contenido de lectura (descripciones).

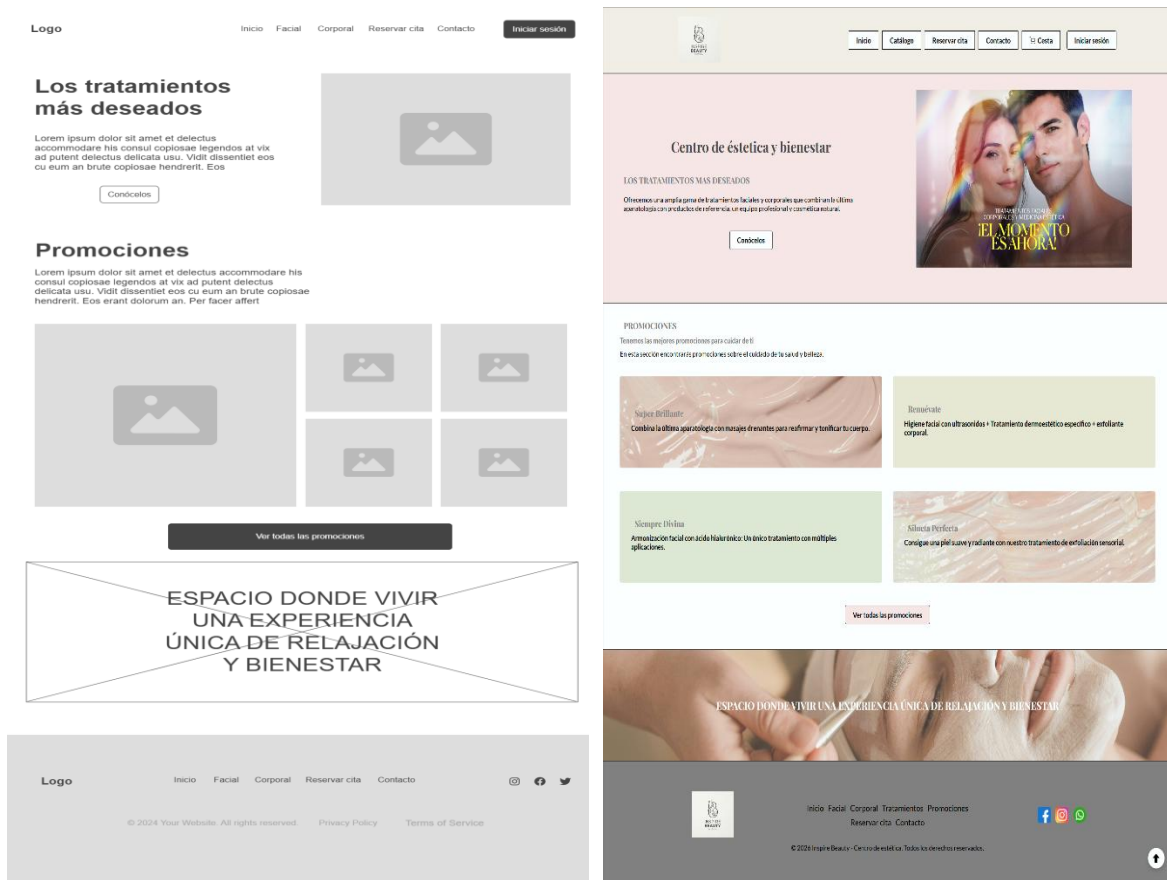


Ilustración 3: Comparativa entre el prototipo inicial (Wireframe) y la implementación final de la página de inicio de Inspire Beauty.

El resto de los bocetos detallados de cada sección de la plataforma pueden consultarse en el **Anexo I** de este documento.

6.3.3. Adaptabilidad y Filosofía Mobile First

Hoy en día, una gran parte del tráfico web proviene de dispositivos móviles. Por ello, la interfaz de Inspire Beauty ha sido diseñada bajo los estándares más estrictos del diseño web responsivo (*Responsive Web Design*), adoptando de manera nativa la filosofía **Mobile First**. Esto significa que las reglas base de las hojas de estilo (CSS) están escritas pensando primero en las pantallas más pequeñas; solo a medida que la pantalla crece, se van aplicando nuevas reglas que reestructuran el contenido para aprovechar el espacio extra.

En lugar de depender de frameworks de interfaz de usuario externos (como Bootstrap), que pueden añadir un gran volumen de código innecesario, se ha desarrollado un **sistema de rejilla (Grid) personalizado** y ligero.

Este sistema propio se sustenta en dos pilares:

1. Uso Intensivo de Flexbox (.row): Se ha creado un contenedor principal .row con la propiedad display: flex; y flex-wrap: wrap;. Esto permite que los elementos hijos fluyan dinámicamente y se adapten al espacio disponible sin desbordamientos, garantizando una alineación perfecta de las tarjetas de tratamientos o productos independientemente del tamaño del dispositivo.

2. Sistema de Columnas Base-12 y Media Queries Estratégicas: Se han definido clases fraccionales (desde .col-1 hasta .col-12) que controlan el ancho de los elementos mediante porcentajes (por ejemplo, width: 50%; para .col-6). Mediante el uso de @media queries estratégicas (como @media (min-width: 576px) para tablets y @media (min-width: 768px) para pantallas de escritorio), se altera el comportamiento de estas columnas:

- **En dispositivos móviles (por defecto):** Todos los contenedores ocupan el 100% del ancho de la pantalla (width: 100%;), apilando el contenido de manera vertical (en una única columna) para facilitar el desplazamiento táctil (*scroll*) y evitar que la información se vea diminuta.
- **En pantallas de escritorio (Breakpoints superiores):** Las *media queries* entran en acción, reorganizando los contenedores para distribuirlos en varias columnas (por ejemplo, dividiendo un banner en un 50% para imagen y un 50% para texto), optimizando la lectura en monitores anchos.

Este desarrollo a medida de la cuadrícula garantiza una experiencia de usuario óptima, un rendimiento web superior por la ligereza del código y asegura que la estructura de la información se adapte de manera fluida y elegante a cualquier resolución de pantalla o dispositivo de acceso.

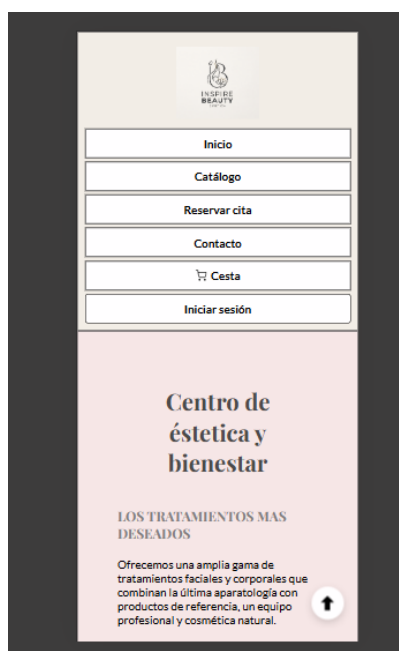


Ilustración 4: Comprobación del diseño adaptativo (Responsive) en entorno de escritorio y dispositivo móvil.

- Interactividad (Feedback Visual): Uso de pseudo clases como :hover y transiciones suaves en botones y elementos de navegación para mejorar la experiencia de usuario y confirmar las acciones de forma visual.

6.3.4. Interactividad Nativa (Arquitectura sin dependencias JavaScript)

Uno de los logros técnicos más destacables en el desarrollo de la capa Frontend ha sido la implementación de soluciones de interactividad compleja apoyándose exclusivamente en CSS3 nativo. Al prescindir de librerías externas o scripts de JavaScript para el comportamiento visual básico de la interfaz, se ha conseguido reducir drásticamente el peso de la página, mejorar la velocidad de carga (Performance) y eliminar posibles fallos en navegadores que tengan la ejecución de scripts bloqueada.

A continuación, se detallan los elementos clave desarrollados bajo este paradigma:

1. Menú Desplegable (Dropdown) mediante Pseudo-clases CSS: Habitualmente, los submenús de navegación requieren JavaScript para gestionar los eventos de clic o mouse-enter. En este proyecto, la funcionalidad completa del menú "Catálogo" se ha resuelto mediante el uso avanzado de selectores CSS. Se estableció un contenedor oculto (display: none;) anclado a un elemento padre. Utilizando la pseudo-clase :hover combinada con el selector de descendencia (.dropdown:hover .menu-desplegable), el submenú cambia su estado dinámicamente a display: block; únicamente cuando el puntero del usuario sobrevuela el elemento superior. A esto se le han añadido detalles de elevación mediante sombras (box-shadow) para separar visualmente el menú del fondo.

2. Sistema de Feedback Táctil y Estados Dinámicos (:hover y Transiciones): Toda la interfaz está diseñada para responder de forma fluida a las acciones del usuario. Los botones de llamada a la acción (como "Reservar cita" o "Pagar") cuentan con contrastes invertidos que se activan al pasar el cursor. Para que este cambio de color no sea abrupto, se han implementado propiedades nativas de transition (ej. transition: 0.3s ease;), lo que genera una interpolación suave entre estados, aumentando la sensación de calidad y diseño premium.

3. Elementos Dinámicos y Gráficos Ligeros (SVG Integrados): En áreas interactivas sensibles, como la visualización del estado de la "Cesta", se ha integrado un icono SVG en línea (inline SVG) en lugar de depender de imágenes pesadas o librerías de fuentes de iconos de terceros (como FontAwesome). El uso de gráficos vectoriales integrados en el código HTML permite manipular su tamaño y color mediante CSS directamente, garantizando una visualización nítida en pantallas de alta densidad de píxeles (Retina) sin sumar peticiones HTTP extra al servidor web.

4. Interactividad Visual (Scroll To Top): La web cuenta con una flecha interactiva en la esquina inferior que permite al usuario volver rápidamente a la cabecera. De nuevo, su comportamiento visual (un sutil desplazamiento vertical y un oscurecimiento del fondo al hacer hover) está gestionado enteramente por transformaciones CSS (transform: translateY(-5px);).

En conjunto, estas prácticas reflejan una filosofía de desarrollo Frontend moderna y optimizada, donde el motor de renderizado del navegador (CSS) absorbe la carga gráfica, liberando recursos y garantizando una experiencia de usuario (UX) sumamente pulida.

6.3.5 Usabilidad e Iconografía

Para garantizar que los usuarios encuentren rápidamente lo que buscan y realicen las acciones clave de la plataforma (como reservar una cita o comprar un tratamiento), la estructura de la web se ha diseñado bajo los principios de reducción de la carga cognitiva. En lugar de abrumar al visitante con múltiples opciones y menús complejos, la navegación se ha simplificado y jerarquizado en un flujo lógico, limpio y directo.

La arquitectura del sitio se estructura en cuatro grandes ejes de interacción:

1. Home (Página de Aterrizaje y Escaparate): La página principal actúa como el escaparate visual del centro. Su objetivo no es dar toda la información de golpe, sino seducir al usuario. Se centra en la presentación inmediata de los servicios estrella y las promociones destacadas mediante tarjetas visuales. Incluye llamadas a la acción (Calls to Action) claras que dirigen el tráfico orgánico hacia el catálogo completo o hacia la reserva directa.

2. Catálogos (Segmentación del Servicio): Para evitar la sobrecarga de información, los servicios del centro se han categorizado lógicamente de forma que el cliente pueda filtrar según sus necesidades.

- Facial y Corporal: Presentan información detallada de tratamientos agrupados por áreas de cuidado.

- Tratamientos y Promociones: Listados completos con la posibilidad de consultar precios y añadir elementos dinámicamente a la cesta de la compra.

3. Conversión (El Embudo de Ventas): La meta final de la aplicación web es generar clientes (reservas o ventas). Por ello, los puntos de conversión están altamente optimizados para minimizar fricciones o dudas durante el proceso.

- Contacto: Un formulario claro y accesible para resolver dudas previas.

- Cesta y Pasarela de Pago: Un proceso estructurado (Checkout) que guía al cliente paso a paso, solicitando únicamente los datos de facturación y pago estrictamente necesarios, y devolviendo feedback visual inmediato (mensajes de éxito o error) tras cada acción.

4. Acceso Privado (Control y Fidelización): El sistema integra una capa de seguridad y privacidad, dividiendo la experiencia post-conversión en dos dashboards (paneles de control) totalmente segregados y protegidos mediante roles de usuario:

- Panel de Cliente: Permite al usuario consultar su histórico de citas, ver sus compras realizadas y gestionar su perfil personal de forma autónoma.

- Panel de Administrador: Un área de uso exclusivo para el personal clínico. Desde aquí, el staff puede visualizar el calendario global de trabajo, auditar los ingresos en caja, generar citas manuales y gestionar la base de datos de pacientes, todo desde una interfaz unificada y de fácil acceso.

7. TECNOLOGÍA UTILIZADA

Para el desarrollo del proyecto "Inspire Beauty", se ha optado por una filosofía de programación basada en tecnologías nativas (Vanilla). La decisión de prescindir de grandes frameworks de terceros (como Laravel en Backend o React/Bootstrap en Frontend) persigue

un doble objetivo formativo y académico: por un lado, poner en práctica y afianzar los conocimientos fundamentales adquiridos durante el Grado Superior de Desarrollo de Aplicaciones Web (DAW); y por otro, comprender el funcionamiento interno de una aplicación web desde sus bases lógicas antes de dar el salto a herramientas automatizadas. Como resultado secundario de este enfoque artesanal, se ha logrado construir una aplicación funcional, sumamente ligera y sin exceso de código innecesario.

7.1. Backend y Persistencia

El núcleo lógico de la aplicación, la gestión de reglas de negocio y el acceso a los datos han sido desarrollados íntegramente por Inmaculada Hortelano. Para cumplir con el objetivo académico de afianzar conocimientos, se ha evitado el uso de frameworks prefabricados, construyendo una arquitectura sólida desde cero apoyada en las siguientes tecnologías:

1. PHP 8 (Vanilla PHP) y Patrón MVC: El lenguaje principal del lado del servidor es PHP en su versión 8. Para estructurar el código de manera ordenada y mantenible, se ha implementado de forma manual el Patrón de Arquitectura MVC (Modelo-Vista-Controlador). Esta separación de responsabilidades garantiza que el código sea modular:

- **Modelos (/app/Models):** Clases encargadas exclusivamente de interactuar con la base de datos (por ejemplo, User.php o Appointment.php), representando las entidades del dominio y ejecutando las sentencias SQL.
- **Vistas (/app/Views):** Archivos que contienen la estructura HTML. Son completamente "tontas", es decir, no contienen lógica de negocio, limitándose a mostrar las variables que reciben del controlador.
- **Controladores (/app/Controllers):** Actúan como el cerebro del sistema. Reciben las peticiones del usuario, piden información a los modelos, aplican las comprobaciones necesarias (ej. verificar si una fecha de cita está libre) y, finalmente, renderizan la vista adecuada con los datos procesados.

2. Enrutador Personalizado (Custom Router) y Controlador Frontal: En lugar de tener múltiples archivos PHP sueltos a los que el usuario accede directamente, se ha implementado el patrón de diseño Controlador Frontal (Front Controller). Todo el tráfico de la web pasa por un único punto de entrada: el archivo public/index.php. A partir de ahí, una clase Router programada a medida se encarga de analizar la URL solicitada. Este enrutador es capaz de mapear URLs amigables (limpias, sin extensiones .php ni parámetros complejos en la barra de direcciones, como /reservar o /cesta) y dirigir las al método específico del

controlador correspondiente, separando además las peticiones según el método HTTP utilizado (GET para pedir vistas, POST para enviar formularios).

3. Base de Datos SQLite y Conexión PDO: Para la persistencia de datos (almacenamiento de clientes, usuarios, citas, tratamientos y ventas) se ha elegido SQLite. Al ser un motor de base de datos serverless (que guarda toda la información en un único archivo físico dentro del proyecto), resulta ser la tecnología óptima para el entorno de evaluación de un Trabajo de Fin de Ciclo, ya que facilita enormemente el despliegue y la portabilidad del código entre diferentes ordenadores sin necesidad de configurar servidores de bases de datos complejos como SQLite.

Toda la comunicación entre PHP y SQLite se realiza a través de la interfaz PDO (PHP Data Objects). Esta extensión proporciona una capa de abstracción de acceso a datos robusta y orientada a objetos. Su principal ventaja técnica en este proyecto ha sido la posibilidad de ejecutar todas las operaciones en la base de datos a través de consultas preparadas (Prepared Statements), lo cual optimiza el rendimiento y neutraliza de raíz los ataques de seguridad por inyección SQL.

7.2. Integración y Seguridad

Garantizar la privacidad de los datos de los clientes y salvaguardar la integridad administrativa del centro clínico ha sido una prioridad transversal durante todo el desarrollo. Al prescindir de librerías de terceros que resuelvan estos problemas "mágicamente", se han tenido que implementar manualmente diversas capas de seguridad estándar de la industria en el entorno de back-end:

1. Gestión de Sesiones y Control de Acceso (Routing Security): Dado que el protocolo HTTP es "sin estado" (stateless), la persistencia de la identidad del usuario a lo largo de su navegación se ha gestionado mediante las variables superglobales \$_SESSION de PHP. El sistema no solo verifica si un usuario ha iniciado sesión, sino que aplica una estricta jerarquía de roles (Cliente vs. Administrador). Los Controladores responsables de áreas sensibles (como el Dashboard del administrador, el calendario global o la gestión de pacientes) incluyen verificaciones de seguridad en sus constructores. Si un usuario anónimo o un cliente sin privilegios intenta forzar el acceso introduciendo la URL manualmente, el enrutador bloquea la petición y lo redirige automáticamente a la página de login o portada, asegurando la inaccesibilidad de la zona restringida.

2. Criptografía y Hashing de Contraseñas: Cumpliendo con los estándares de ciberseguridad, bajo ninguna circunstancia se almacenan contraseñas en texto plano en la

base de datos. Para el registro de usuarios, se utiliza la función nativa de PHP `password_hash()`, la cual implementa el robusto algoritmo de cifrado asimétrico Bcrypt. Este algoritmo genera automáticamente un salt (secuencia aleatoria) único para cada contraseña, lo que hace inviable descifrarlas mediante ataques de fuerza bruta o "tablas arcoíris" (rainbow tables). Durante el inicio de sesión, la validación se realiza de forma segura mediante `password_verify()`. Se contempló además una lógica específica para los clientes creados físicamente en mostrador, permitiendo crear perfiles inactivos en la web que puedan ser reclamados por el cliente posteriormente sin comprometer la seguridad.

3. Sanitización de Datos y Prevención de Ataques (SQLi y XSS): Para garantizar la inmunidad del servidor ante ataques maliciosos, se han blindado tanto las entradas (Inputs) como las salidas (Outputs) de datos:

- **Inyección SQL (SQLi):** Tal y como se abordó en la capa de persistencia, absolutamente todas las consultas a la base de datos se procesan a través de sentencias preparadas de PDO, separando estrictamente la estructura SQL de los datos enviados por el usuario.
- **Cross-Site Scripting (XSS):** En la capa de Vistas (Frontend), cualquier dato extraído de la base de datos o enviado por un usuario (por ejemplo, los mensajes del formulario de contacto) se procesa a través de la función `htmlspecialchars()` antes de ser renderizado en pantalla. Esto convierte caracteres especiales en entidades HTML inofensivas, impidiendo que código JavaScript malicioso pueda ser ejecutado en el navegador de otros usuarios o administradores.

4. Integración Arquitectónica: Simulador de Correo Interno (EmailService): A nivel de integración, el proyecto requería la capacidad de notificar a los clientes sobre sus compras y reservas. Dado que el proyecto debía ser fácilmente auditable en un entorno local para su evaluación académica —y para evitar la dependencia de servicios SMTP externos de pago o configuraciones de red complejas—, se desarrolló una solución de integración a medida: el Simulador de Correos Electrónicos. El componente EmailService intercepta los envíos del sistema, inyecta las variables transaccionales (nombres, importes, IDs de pedido) en plantillas HTML y genera un archivo físico único en el almacenamiento interno del servidor (`/storage/emails/`). Posteriormente, genera una URL dinámica para visualizar el resultado en el navegador web, logrando una simulación perfecta del flujo de notificaciones de forma 100% autocontenida.

7.3. Frontend: HTML5, CSS3 y JS Nativo

La capa de presentación web, desarrollada y diseñada por Irene González, constituye el punto de interacción directa con el cliente. Siguiendo la premisa central del proyecto de prescindir de utilidades externas (como Bootstrap, Tailwind o jQuery) con el fin de afianzar los conocimientos del grado superior, todo el frontend se ha construido empleando el ecosistema nativo o Vanilla. Esto ha permitido tener un control granular sobre cada píxel y comportamiento de la pantalla, dando como resultado un rendimiento de carga excepcional.

Las tecnologías empleadas en este nivel se dividen en tres áreas fundamentales:

1. Estructuración con HTML5 Semántico: Lejos de utilizar la etiqueta genérica <div> para estructurar la totalidad de los documentos, el esqueleto de la aplicación se ha construido empleando etiquetas semánticas introducidas en el estándar HTML5 (tales como <header>, <nav>, <main>, <section>, <article> y <footer>). Esta decisión técnica trasciende lo puramente estético y cumple dos funciones clave:

- Accesibilidad (A11y): Permite a los navegadores y a los lectores de pantalla para personas con discapacidad visual interpretar correctamente las diferentes secciones de la web y su nivel de relevancia.
- Optimización para Motores de Búsqueda (SEO): Ayuda a los algoritmos de indexación (como Google) a comprender la estructura y jerarquía de la información, mejorando el posicionamiento orgánico del centro de estética en internet.

2. Maquetación y Diseño con CSS3 Puro: Todo el apartado visual (colores, tipografías, cuadrículas y comportamiento responsive) reside en hojas de estilo en cascada nativas. Se han explotado al máximo las capacidades modernas de CSS3:

- Arquitectura mediante Custom Properties: Uso intensivo de variables (:root) para centralizar la paleta de colores y simplificar el mantenimiento global del diseño.
- Flexbox y Media Queries: Desarrollo de un sistema de rejilla fluido bidimensional que permite reorganizar los contenedores bajo el paradigma Mobile First, garantizando una visualización perfecta en cualquier tamaño de dispositivo.
- Interactividad Autónoma: Delegación de tareas interactivas complejas (como la aparición de submenús desplegados, la superposición de sombras en elementos al pasar el ratón o la creación de menús estáticos) enteramente al motor de renderizado CSS (usando :hover y transiciones), aliviando la carga de procesamiento del lado del cliente.

3. Interactividad Estratégica con JavaScript Nativo (Vanilla JS): El uso de JavaScript se ha limitado y focalizado de manera estratégica en aquellos procesos donde CSS no tiene alcance, concretamente en la mejora de la Experiencia de Usuario (UX) a través de la lógica del lado del cliente. Al no utilizar librerías pesadas como jQuery, la ejecución de los scripts es prácticamente instantánea. Su implementación más destacada se encuentra en las vistas de formulario y en la pasarela de pago (Checkout). Mediante el uso de escuchadores de eventos (EventListeners), JavaScript actúa en tiempo real capturando lo que el usuario teclea para formatear dinámicamente los datos (por ejemplo, inyectando automáticamente espacios cada cuatro dígitos en el número de la tarjeta de crédito, o insertando la barra divisoria en la fecha de caducidad MM/AA). Esto previene errores de introducción de datos por parte del cliente antes siquiera de que el formulario viaje hacia el servidor. Se ha implementado tecnología AJAX mediante la API fetch nativa de JavaScript para realizar consultas asíncronas al servidor (como la disponibilidad de horas en las reservas) sin necesidad de recargar la página. Esto permite que el cliente pueda visualizar las horas disponibles para reservar su cita (contabilizando también la duración del tratamiento seleccionado) evitando solapamientos entre las citas

7.4. Entorno de Desarrollo y Extensiones (VS Code)

Para la escritura y gestión del código fuente, hemos utilizado unánimemente **Visual Studio Code (VS Code)**. Su elección se fundamenta en su naturaleza liviana y su amplísimo ecosistema de extensiones, el cual ha permitido personalizar el editor para agilizar el flujo de trabajo conjunto y cubrir las necesidades específicas de los roles de Frontend y Backend.

El entorno de desarrollo se ha complementado con las siguientes herramientas clave:

- **Entorno de Servidor Local:** Para procesar el código de servidor (PHP) y las consultas a la base de datos (SQLite), el proyecto se ha desplegado haciendo uso de un servidor local (mediante herramientas como XAMPP o el servidor nativo CLI de PHP), simulando el comportamiento de un entorno de producción real.
- **PHP Intelephense (Backend):** Extensión fundamental para el desarrollo de la lógica de servidor. Proporcionó a la capa de Backend autocompletado inteligente de código, detección de errores de sintaxis en tiempo real e inspección profunda de clases y métodos (crucial para navegar por la estructura MVC personalizada).
- **SQLite Viewer / Database Client (Backend):** Dado que la persistencia se apoya en SQLite, el uso de extensiones de exploración de bases de datos integradas en VS Code permitió a la responsable de Backend inspeccionar las tablas, lanzar consultas SQL de prueba y auditar las inserciones de datos (como citas o nuevos usuarios) sin tener que abandonar el editor ni utilizar gestores de bases de datos externos pesados.
- **Live Server (Frontend):** Durante las fases iniciales de maquetación, esta extensión resultó vital para la capa visual. Permitió levantar un entorno con recarga en caliente (*hot-reloading*), visualizando en tiempo real cualquier ajuste de píxeles o colores en el navegador antes de su integración definitiva con PHP.

- **CSS Peek (Frontend):** Teniendo en cuenta la extensa arquitectura de variables creada en la hoja de estilos, esta extensión permitió saltar instantáneamente desde el nombre de una clase en el archivo HTML o PHP directamente hacia su regla de definición en el archivo .css, agilizando el mantenimiento del diseño.
- **Prettier (Code Formatter):** Para mantener un estilo de código unificado entre ambas desarrolladoras, esta extensión se encargó de formatear automáticamente los archivos al guardarlos (tabulaciones, saltos de línea), garantizando un código limpio y fácil de auditar.
- **Git y GitHub (Control de Versiones y Trabajo Colaborativo):** Dado que el proyecto ha sido desarrollado por dos personas asumiendo roles diferenciados (Frontend y Backend), era indispensable contar con un sistema robusto para el trabajo simultáneo. Se ha utilizado Git como sistema de control de versiones local, y GitHub como plataforma de alojamiento del repositorio remoto. Esto ha permitido trabajar en ramas (branches) separadas, centralizar el código en la nube, mantener un historial seguro de las versiones (commits) y fusionar (merge) los desarrollos de ambas partes mitigando el riesgo de pérdida de código o sobrescritura de archivos.

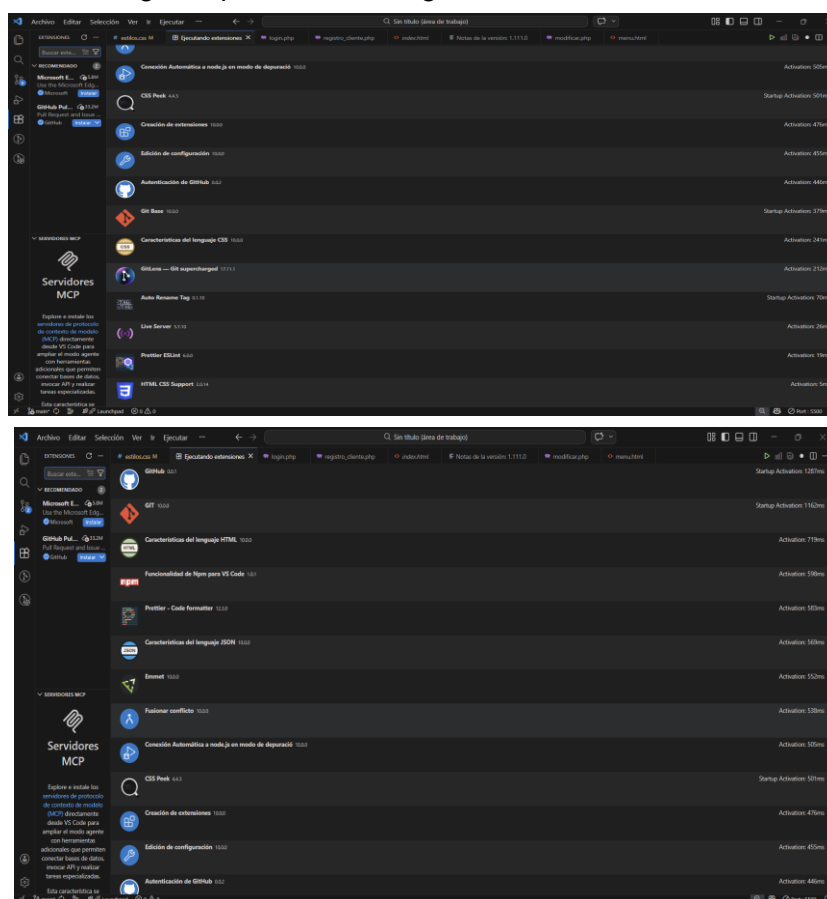


Ilustración 5: Configuración del entorno de desarrollo y extensiones activas en Visual Studio Code para el proyecto Inspire Beauty.

8. Metodología y Planificación

Para garantizar el éxito del desarrollo de "Inspire Beauty", adoptamos un enfoque de trabajo iterativo inspirado en las Metodologías Ágiles. Dado que el proyecto requería la coordinación paralela de dos roles distintos (Frontend y Backend), era fundamental establecer un marco de trabajo que dotara de transparencia al progreso del proyecto y evitara bloqueos mutuos.

Entre los hitos más significativos del proyecto destacan:

1. Definición de Wireframes y arquitectura de la información.
2. Implementación de Estilos base y sistemas de cuadrícula.
3. Optimización para Responsive Design.
4. Integración final de formularios y validaciones.

8.1. Planificación con Trello y Filosofía Kanban

Como herramienta central para la gestión del proyecto, se seleccionó Trello, apoyándose en la filosofía visual Kanban. Esta metodología permite mapear el flujo de trabajo en un tablero digital, dividiendo la carga total del proyecto en pequeñas tareas asumibles (Tarjetas) y moviéndolas a través de diferentes fases de estado.

El tablero de trabajo se estructuró en las siguientes columnas principales:

Pendiente (To Do / Backlog): Un repositorio general donde se volcaron todos los requisitos del proyecto tras la fase de análisis inicial. Aquí se encontraban tarjetas como "Crear base de datos", "Diseñar maqueta del carrito" o "Lógica de inicio de sesión".

En Proceso (In Progress): Tareas en las que las desarrolladoras se encontraban trabajando activamente en un momento dado. Esta columna tenía un límite implícito de tareas simultáneas (WIP Limit) para asegurar el foco y evitar dejar desarrollos a medias.

Hecho (Done): El histórico de logros. Tareas finalizadas, testeadas y listas para ser integradas en la rama principal del repositorio de GitHub.

Beneficios de esta implementación:

División de Tareas por Rol: Cada tarjeta de Trello fue asignada explícitamente a Inmaculada (Backend) o a Irene (Frontend). Esto clarificó las responsabilidades individuales de cada día.

Prevención de Cuellos de Botella: La metodología Kanban permitió trabajar de forma asíncrona pero coordinada. Por ejemplo, mientras el Backend construía las rutas y el modelo de datos de un apartado, el Frontend podía maquetar simultáneamente la vista estática usando datos falsos (mocks). Una vez ambas tarjetas pasaban a "Hecho", se procedía a su integración.

Comunicación Centralizada: Las tarjetas de Trello no solo servían como recordatorios, sino como centro de comunicación, permitiendo adjuntar capturas de pantalla, detallar requisitos técnicos y añadir checklists internas para asegurar que ninguna funcionalidad clave quedaba en el olvido.

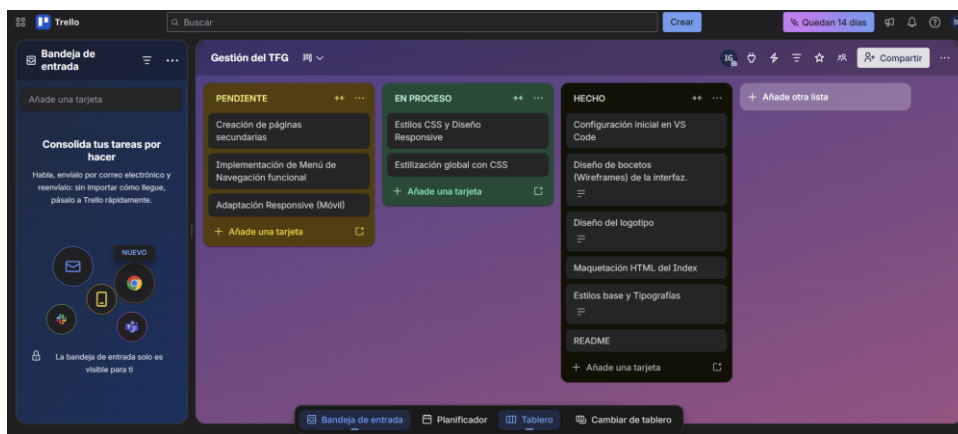


Ilustración 6: Estado del tablero Trello durante la fase de desarrollo del Frontend.

8.2. Control de Versiones con GitHub

Dada la naturaleza colaborativa del proyecto y la necesidad de integrar continuamente el trabajo de las dos desarrolladoras, prescindir de un sistema de control de versiones suponía un riesgo inaceptable de pérdida de código o sobrescritura accidental. Por ello, se implementó Git como sistema de control de versiones local, y GitHub como plataforma de alojamiento del repositorio en la nube.

El flujo de trabajo (workflow) establecido en GitHub se fundamentó en las siguientes buenas prácticas:

1. Estrategia de Ramificación (Branching): Para mantener la estabilidad del proyecto, el código principal (la versión funcional) se alojaba siempre en la rama principal (main). El desarrollo de nuevas funcionalidades se llevaba a cabo creando ramas paralelas (Branches). Esto permitió a Inmaculada trabajar en la lógica del servidor (por ejemplo, en una rama feature/backend-auth) al mismo tiempo que Irene diseñaba la vista correspondiente (en una rama feature/frontend-login), asegurando un desarrollo simultáneo y aislado que nunca rompía la aplicación central.

2. Trazabilidad y Commits Descriptivos: Cada avance significativo en el código se registraba mediante un Commit. Se estableció como norma interna la redacción de mensajes de commit semánticos, claros y descriptivos (ej. "Añadida validación de formulario de reserva" en lugar de "Cambios"). Esta práctica profesional garantizó un historial del proyecto totalmente trazable, permitiendo auditar la evolución de la web o revertir el sistema a un punto de control anterior de forma segura en caso de error crítico.

3. Integración Continua y Resolución de Conflictos: El mayor desafío de coordinar Frontend y Backend reside en el momento de juntar ambas partes. Utilizando GitHub, el proceso de volcar los desarrollos de las ramas secundarias hacia la rama main se realizaba mediante operaciones de fusión (Merges). Si ambas desarrolladoras habían modificado las mismas líneas de un archivo simultáneamente (por ejemplo, al integrar una vista HTML con las etiquetas de PHP), la plataforma detectaba la discrepancia, obligando a realizar una resolución estructurada de conflictos antes de autorizar la fusión. Esto garantizó que la integración de la capa de presentación y la del servidor fuera siempre fluida, controlada y libre de errores.

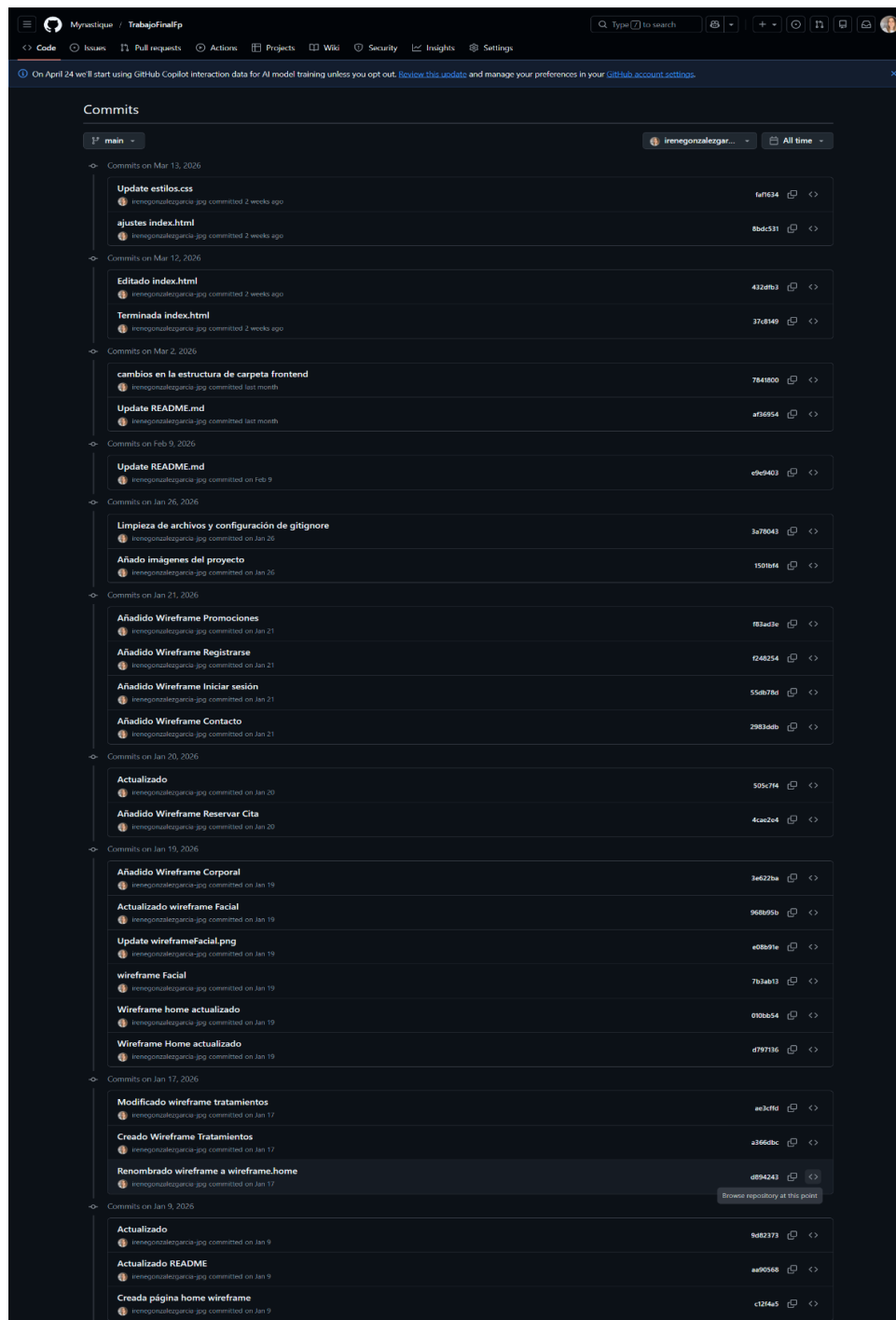


Ilustración 7: Historial de commits en GitHub

8.3. Cronograma de Ejecución

Para asegurar que el proyecto se desarrollase dentro de los plazos estipulados por el calendario académico, evitando solapamientos y retrasos de última hora, el ciclo de vida del software se dividió lógicamente en cuatro grandes fases secuenciales:

Fase 1: Análisis y Toma de Requisitos: El proyecto comenzó con la definición clara y estricta de los objetivos del sistema. En esta etapa de planificación arquitectónica, se determinó el alcance de la aplicación (qué funcionalidades eran prioritarias para el Producto Mínimo Viable y cuáles eran secundarias). Se analizaron los flujos de usuario (el proceso de compra paso a paso, la reserva de citas) y se diseñó sobre papel la estructura lógica de la base de datos (Modelo Entidad-Relación) necesaria para soportar dichos procesos.

Fase 2: Diseño de Interfaz y Maquetación (Capa de Presentación): Una vez definido el alcance, el esfuerzo se centró en la identidad visual y la experiencia de usuario (UX/UI). Esta fase comenzó con la concepción estética de "Inspire Beauty", seguida de la codificación estructural. Se redactaron los archivos base de HTML5 semántico y se construyó la arquitectura centralizada de CSS3. El objetivo de esta fase fue conseguir unas vistas completamente funcionales a nivel de diseño, responsivas e interactivas (hovers, menús) con datos simulados estáticos, dejando el "esqueleto" listo para cobrar vida.

Fase 3: Desarrollo Técnico e Integración (Lógica de Servidor): Con la maquetación estabilizada, el foco se trasladó a la ingeniería del Backend. Se programó la arquitectura interna basándose en el patrón MVC (Modelo-Vista-Controlador), implementando el enrutador personalizado y configurando la conexión PDO con la base de datos SQLite. Durante esta fase se produjo el gran hito del proyecto: la integración fluida de ambas capas. Las vistas estáticas de la Fase 2 fueron transformadas en plantillas dinámicas .php, permitiendo la inyección de los datos reales del servidor (como el cálculo del carrito o la disponibilidad del calendario).

Fase 4: Pruebas (Testing): Depuración y Entrega La última fase del cronograma se dedicó íntegramente a asegurar la calidad del producto final (Quality Assurance). Se llevaron a cabo exhaustivas sesiones de depuración (Debugging) para cazar conflictos visuales de CSS y subsanar posibles brechas de seguridad (validaciones de formularios, control de acceso por roles). Finalmente, una vez garantizada la estabilidad del código, se procedió al congelamiento del desarrollo (Code Freeze) para concentrar los esfuerzos en la redacción detallada de la presente Memoria Técnica, documentando las decisiones y tecnologías que han dado forma a la plataforma.

9. Manual de instalación

Debido a que el proyecto "Inspire Beauty" está construido utilizando SQLite como motor de base de datos, su portabilidad es máxima. La base de datos está autocontenida en los archivos del proyecto, por lo que no es necesario instalar, configurar ni levantar gestores externos como MySQL o phpMyAdmin.

Para evaluar el proyecto en un entorno local, existen dos metodologías de despliegue:

Opción A: Despliegue rápido mediante el Servidor Integrado de PHP (Recomendado)

Esta es la forma más limpia y moderna de arrancar el proyecto, ideal para entornos de desarrollo, ya que no requiere la instalación de software pesado.



Requisito previo: Tener instalado PHP 8 (o superior) en el sistema operativo y añadido a las variables de entorno (Path).

Clonar el repositorio: Descargar o clonar la carpeta del proyecto TrabajoFinalFp en cualquier ubicación del ordenador.

Abrir la terminal: Navegar a través de la consola de comandos hasta la carpeta raíz del proyecto.

Iniciar el servidor: Ejecutar el siguiente comando para levantar el servidor interno de PHP apuntando directamente al controlador frontal (public): `php -S localhost:8000 -t public`

Acceso: Abrir cualquier navegador web y dirigirse a `http://localhost:8000`.

Opción B: Despliegue tradicional mediante XAMPP / Laragon

Si el usuario evaluador prefiere utilizar un entorno de servidor web clásico completo:

Instalación: Instalar un entorno de servidor como XAMPP o Laragon.

Ubicación de archivos: Copiar la carpeta completa del proyecto TrabajoFinalFp dentro del directorio público del servidor (generalmente `C:\xampp\htdocs` en XAMPP o `C:\laragon\www` en Laragon).

Iniciar servicios: Abrir el panel de control de XAMPP e iniciar únicamente el servicio Apache (el servicio MySQL debe permanecer apagado, ya que el proyecto utiliza SQLite).

Acceso: Abrir el navegador y acceder a la ruta local. (Nota: Dependiendo de la configuración del servidor Apache, es posible que la URL de acceso sea `http://localhost/TrabajoFinalFp/public` para alcanzar el archivo `index.php` principal del enrutador).

10. Manual de Usuario (Perfil Cliente y Gestión Visual)

Este manual detalla el flujo de interacción y las funcionalidades a disposición de los usuarios de la plataforma, garantizando una experiencia de navegación intuitiva y orientada a la conversión.

10.1. Acceso y Navegación Principal

Al acceder a la plataforma, el usuario es recibido por la Página de Inicio (*Home Page*). Esta vista actúa como el escaparate principal del centro, diseñada estratégicamente para mostrar los servicios estrella sin sobrecargar cognitivamente al visitante. Desde este punto de partida, la navegación se estructura a través de una barra superior permanente (*Sticky Header*) que actúa como centro de mando, adaptándose fluidamente tanto a pantallas de escritorio como a dispositivos móviles. Las principales vías de exploración que ofrece la navegación son:

Menú Desplegable de Catálogo: Agrupa de forma limpia la oferta del centro. Al interactuar con él, el usuario puede segmentar su búsqueda accediendo a listados específicos de tratamientos "Faciales", "Corporales", ver la sección completa de "Tratamientos" o descubrir las "Promociones" activas.

Acciones de Conversión Directas (CTAs): Botones destacados en la navegación principal como "Reservar Cita" (que conduce directamente al calendario interactivo) y "Contacto" (para resolución de dudas).

Estado en Tiempo Real (Cesta y Sesión): La cabecera es dinámica. El usuario puede visualizar en todo momento el icono de su "Cesta" (cuyo contador se actualiza instantáneamente si contiene productos) y el botón de "Iniciar sesión". Si el usuario ya está autenticado, la cabecera cambiará para darle la bienvenida (ej. "Hola, María") y le ofrecerá acceso directo a su panel privado o la opción de cerrar sesión de forma segura.

10.2. Uso del Formulario de Contacto y Asistencia

Para aquellos usuarios que requieran asistencia directa, tengan dudas sobre algún tratamiento o deseen realizar una consulta genérica antes de reservar, la plataforma dispone de una sección dedicada de Contacto, accesible directamente desde la barra de navegación principal y sin necesidad de iniciar sesión.

El proceso de comunicación se ha diseñado centrándose en la prevención de errores y la inmediatez de respuesta visual:

1. **Interfaz Clara y Requisitos:** El usuario se encuentra con un formulario limpio y estructurado que solicita únicamente la información imprescindible (Nombre, Correo electrónico y el Mensaje de la consulta). Esto reduce la fricción y fomenta que el usuario complete el envío.
2. **Validación en Tiempo Real (Prevención de Errores):** Antes de que el mensaje viaje hacia los servidores del centro, el navegador del usuario (gracias a la lógica implementada en Frontend) evalúa instantáneamente que los campos obligatorios no estén vacíos y verifica que la estructura del correo electrónico introducido sea válida (ej. que contenga un símbolo '@' y un dominio). Si el sistema detecta una anomalía, detiene el envío y muestra una alerta visual proactiva (mediante los atributos required de HTML5), indicando al usuario exactamente qué campo debe corregir.
3. **Confirmación de Éxito (Feedback Visual):** Una vez que el formulario supera las validaciones y es procesado con éxito por el servidor (Backend), el sistema recarga la vista mostrando un mensaje de confirmación claro y en tono positivo. Este feedback es vital en la Experiencia de Usuario (UX), ya que le otorga al cliente la tranquilidad absoluta de que su mensaje ha sido recibido por el equipo de Inspire Beauty y será atendido a la mayor brevedad.

Ilustración 8: Interfaz inicial del formulario de contacto de Inspire Beauty.

Ilustración 9: Ejemplo de validación del lado del cliente ante un formato de correo no válido.

Ilustración 10: Mensaje de confirmación tras el envío exitoso del formulario.

10.3. Formulario de Solicitud de Cita Online

El motor de reservas es una de las funcionalidades más críticas de la plataforma, diseñado para ser completamente autónomo y evitar la sobrecarga de trabajo telefónico al personal de la clínica. Su funcionamiento se basa en las siguientes etapas:

Requisito de Autenticación: Por motivos de seguridad y control de agenda, el sistema exige que el cliente disponga de una cuenta activa para realizar una reserva. Si un usuario anónimo intenta acceder a /reservar, el sistema lo intercepta amigablemente y lo redirige a

la página de inicio de sesión o registro, garantizando que ninguna cita quede huérfana en la base de datos.

Selección de Fecha y Tratamiento: El usuario accede a un formulario interactivo donde debe indicar la fecha deseada y el tipo de tratamiento. El campo de fecha incluye un selector de calendario (Datepicker) nativo que facilita la navegación visual.

Gestión Dinámica de Disponibilidad: Una vez elegida la fecha, el sistema entra en comunicación con el calendario interno del servidor. Mediante una lógica avanzada, el Backend divide la jornada laboral de la clínica en franjas de 15 minutos, cruza esa información con las citas ya aprobadas en la base de datos, y solo muestra las horas que están estrictamente libres. Esto imposibilita matemáticamente el solapamiento o la doble reserva de citas a la misma hora.

Confirmación y Notificación: Al completar la reserva, el cliente es redirigido a su Panel Privado donde podrá ver el estado de su cita. Simultáneamente, el sistema de integración lanza automáticamente el envío de un correo electrónico simulado con el resumen de la reserva y las políticas de cancelación del centro.

The screenshot shows the 'Panel de Administración' with the 'Grabar Cita Manualmente' form. The form has four main sections: 'Cliente' with a dropdown for 'Nombre de Prueba (prueba@prueba.com)', 'Tratamiento' with a dropdown for 'Drenaje Pielmas Ligeras (40 min)', 'Fecha' with a date picker set to '06/06/2026', and 'Hora' with a dropdown menu. A 'Programar Cita' button is next to the time selection. Below the form is a 'Calendario de Citas' for June 2026, showing a grid of dates from 1 to 30. To the right of the calendar is a list of available time slots from 10:15 to 14:45 in 15-minute increments. A 'Confirmada' button is visible at the bottom right of the time slot list.

Ilustración 11: Gestión dinámica de Disponibilidad (AJAX).

10.5. Navegación por el Catálogo y Cesta de la Compra

El núcleo comercial de "Inspire Beauty" reside en su catálogo de tratamientos y en el flujo de selección de servicios. Esta sección ha sido diseñada simulando los estándares de un e-commerce moderno, facilitando la conversión del visitante en cliente.

1. Exploración del Catálogo (Páginas Informativas): Con el objetivo de no saturar al usuario y ofrecer una experiencia de lectura relajada, la plataforma separa la fase de información de la fase de compra. A través del menú principal, el usuario puede acceder a las páginas de **Catálogo (Facial y Corporal)**. Estas vistas están

diseñadas a modo de "revistas digitales", ofreciendo descripciones extensas y detalladas sobre en qué consiste cada tratamiento, sus beneficios médicos y estéticos, y los resultados esperados, acompañadas de contenido altamente visual.

2. Página Transaccional y Añadido Dinámico (La "Tienda"): Una vez que el usuario se ha informado y tomado la decisión, la plataforma dispone de una vista específica y centralizada llamada "**Tratamientos**", enfocada puramente a la conversión. En esta página, la información se presenta de forma mucho más sintética y directa (formato tarjeta), mostrando únicamente los datos clave: nombre, duración en minutos y precio. Es aquí donde se habilita el botón de acción (*Call to Action*) para "**Añadir a la cesta**". Al interactuar con él, el sistema procesa la petición de forma asíncrona, almacenando el servicio en la sesión del usuario de forma transparente, lo que permite al cliente añadir múltiples tratamientos rápidamente sin interrupciones en su navegación.

3. Feedback Visual Constante (Cabecera Dinámica): Para mantener al usuario informado de su intención de compra, el icono del "Carrito de Compra" ubicado en la barra de navegación principal reacciona en tiempo real. Se ha implementado una lógica que muestra dinámicamente un indicador numérico únicamente cuando existen productos en la cesta (ej. Carrito (2)). Si el usuario vacía la cesta, el contador desaparece de forma elegante para mantener la limpieza visual de la interfaz.



Ilustración 12: Cabecera dinámica – Indicador de productos en la Cesta.

4. Gestión de la Cesta de la Compra: Al hacer clic en el icono del carrito, el usuario accede a la vista de resumen (/cesta). En este panel de control, el cliente tiene visibilidad total sobre su pedido antes de proceder al pago:

- **Desglose de Servicios:** Listado detallado de todos los tratamientos seleccionados.
- **Control de Modificaciones:** Botones integrados para eliminar servicios individuales de la cesta si el cliente cambia de opinión, actualizándose el carrito automáticamente.
- **Cálculo Transparente:** Sumatorio automático que muestra el precio de cada tratamiento y el importe total exacto de la compra, sin costes ocultos de última hora.

Desde esta pantalla, y una vez que el usuario está conforme con su selección, se le ofrece la opción de continuar hacia la **Pasarela de Pago Segura**, lo que inicia el flujo final de conversión.

10.6. Pasarela de Pago Simulada (Checkout) y Tickets Electrónicos

Para culminar el flujo de conversión del e-commerce, se ha desarrollado una pasarela de pago completa. Dado el entorno académico del proyecto y la imposibilidad de vincular la plataforma a TPVs virtuales reales (como Redsys o Stripe) que exijan tarjetas bancarias auténticas a los evaluadores, se ha optado por construir un entorno de simulación de transacciones de alta fidelidad.

El proceso de Checkout se compone de las siguientes fases interactuando entre el Frontend y el Backend:

1. Verificación de Autenticación: Al pulsar "Pagar" desde la vista de la cesta, el enrutador verifica el estado de la sesión. Si el usuario es anónimo, el sistema fuerza una redirección inteligente hacia la página de login o registro, guardando la intención de compra para no perder el contenido de la cesta tras el alta.

2. Formulario de Pago y UX Avanzada: Una vez en el Checkout, el cliente se encuentra con un formulario de tarjeta de crédito meticulosamente diseñado. Para ofrecer una simulación hiperrealista, se ha utilizado JavaScript Nativo que asiste al usuario durante la escritura (Live Formatting):

El campo del número de tarjeta inyecta espacios en blanco automáticamente cada cuatro dígitos.

El campo de caducidad inserta automáticamente la barra divisoria (MM/YY).

Se impide la introducción de letras en campos estrictamente numéricos. Estas validaciones del lado del cliente previenen envíos erróneos y maximizan la sensación de realismo.

3. Pantalla de Procesamiento Asíncrono: Al enviar el pago, el usuario no es redirigido bruscamente al resultado. Para emular el comportamiento de una pasarela bancaria real, la plataforma transita a una pantalla intermedia de procesamiento (/checkout/processing). Esta vista retiene visualmente al usuario durante unos segundos mediante una animación de carga (spinner), simulando el tiempo de respuesta y la comunicación encriptada con la entidad bancaria.

4. Confirmación Transaccional y Ticket Electrónico: Superado el tiempo de simulación, la lógica del servidor entra en acción definitiva:

Calcula el importe total y registra la venta formalmente en la base de datos (tabla sales).

Genera un Número de Pedido único y vacía la cesta del usuario.

Despliega la vista final de éxito (/checkout/success), mostrando un resumen desglosado de lo adquirido.

5. Integración del Simulador de Correos: Paralelamente a la pantalla de éxito, el sistema dispara el servicio interno EmailService. Este componente genera un ticket electrónico (Factura Simplificada) en formato HTML y simula su envío al correo del cliente. Para facilitar la evaluación, la pantalla de éxito incluye un enlace directo y dinámico que permite abrir dicho "correo enviado", visualizando la plantilla de facturación con los datos reales del cliente y los tratamientos comprados.

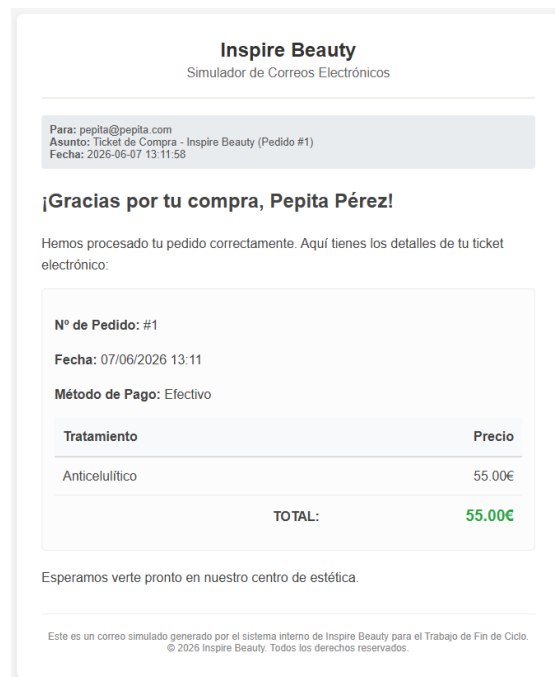


Ilustración 13: Simulación del email de Confirmación de pedido o Ticket digital.

10.7. Área Privada del Cliente (Mi Panel)

Una vez completado el proceso de registro o tras formalizar una compra/reserva, la plataforma dota al usuario de total autonomía mediante un entorno privado y seguro al que se accede desde la cabecera (opción "Mi Panel").

Este espacio (*Dashboard* del Cliente) centraliza toda la información transaccional y de agenda del usuario, protegida por los mecanismos de seguridad de sesión (\$_SESSION), garantizando que cada cliente tenga acceso única y exclusivamente a sus datos personales.

El panel se divide visualmente en dos bloques funcionales principales:

1. Gestión y Seguimiento de Citas: Para evitar llamadas innecesarias a la clínica y automatizar la relación con el paciente, el panel ofrece un listado cronológico de todas las citas solicitadas por el usuario.

- **Estado Dinámico:** Cada cita cuenta con un indicador visual de estado. El usuario puede saber al instante si su cita está *Pendiente* de aprobación por el centro, si ha sido formalmente *Confirmada* o si se encuentra *Cancelada*.
- **Autonomía de Cancelación:** Otorgando el control al usuario y facilitando la gestión de la agenda del centro, el sistema permite al cliente cancelar voluntariamente una cita que se encuentre en estado "Pendiente" o "Confirmada". Al pulsar el botón de cancelación, la lógica del servidor libera automáticamente ese hueco en el calendario, dejándolo disponible para que otro cliente pueda reservarlo en tiempo real.

2. Historial de Pedidos y Compras: En aras de la transparencia comercial, el panel actúa también como un repositorio contable para el usuario. Incluye un historial detallado de todos los tratamientos adquiridos previamente mediante la pasarela de pago de la plataforma. El cliente puede revisar en un solo vistazo el nombre de los tratamientos comprados, la fecha exacta de la transacción y el importe abonado, funcionando a modo de resguardo digital permanente.

11. Manual del Personal de Clínica (Módulo de Administración)

Para que el proyecto sea una solución integral real, no bastaba con ofrecer una interfaz de venta al público. Se ha desarrollado un completo sistema de gestión interna (Back-Office o CRM) de acceso restringido, diseñado exclusivamente para el personal autorizado del centro de estética.

Este entorno se blindo mediante los controladores de PHP, que verifican que la sesión activa corresponda al rol admin, denegando el acceso a cualquier cliente estándar.

11.1. Panel de Control General (*Dashboard* y Métricas)

Nada más iniciar sesión, el administrador aterriza en un panel de mandos o *Dashboard*. Esta vista ha sido diseñada bajo principios de análisis de datos para ofrecer una "fotografía" instantánea del estado de salud del negocio.

En la parte superior, mediante consultas de agregación a la base de datos (SQL), se presentan cuatro indicadores clave de rendimiento (KPIs) en tiempo real:

- **Volumen de Clientes:** Total de usuarios registrados en la plataforma.
- **Gestión Operativa:** Número de citas pendientes de revisión que requieren atención inmediata por parte de recepción.
- **Salud Financiera:** Cálculo dinámico de los ingresos totales generados históricamente a través de la web.
- **Volumen de Ventas:** Cantidad total de tratamientos vendidos.

Por debajo de los indicadores financieros, el administrador dispone de un registro cronológico de **Actividad Reciente**. Esta tabla unifica las acciones más importantes que están sucediendo en la clínica, mostrando las últimas citas solicitadas por los clientes y las ventas cerradas más recientes, permitiendo al personal llevar un control exhaustivo del día a día sin tener que navegar por múltiples apartados.

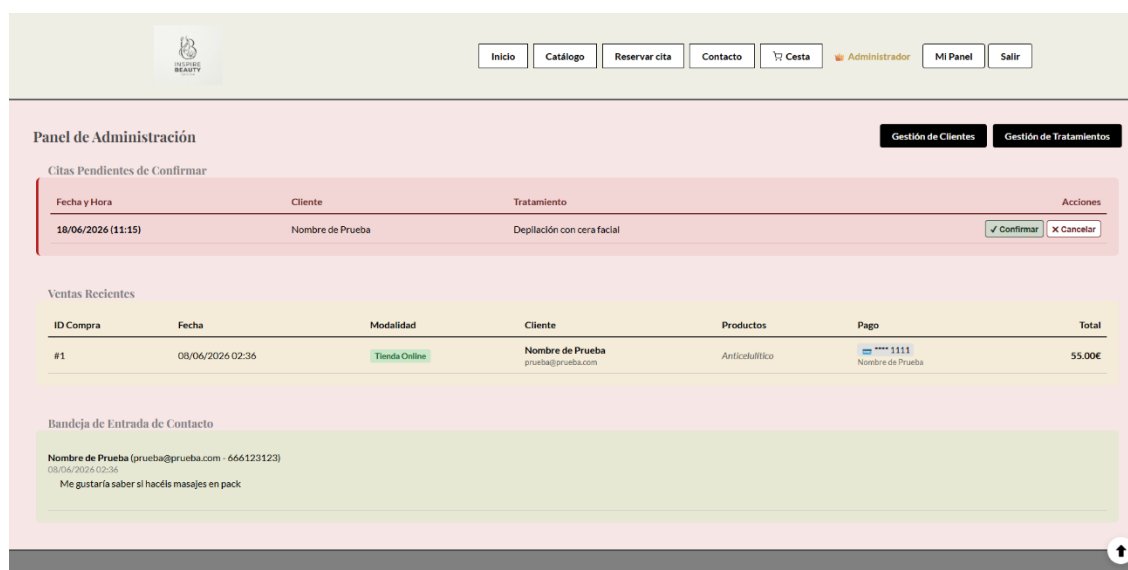


Ilustración 14: Vista del panel Administrador

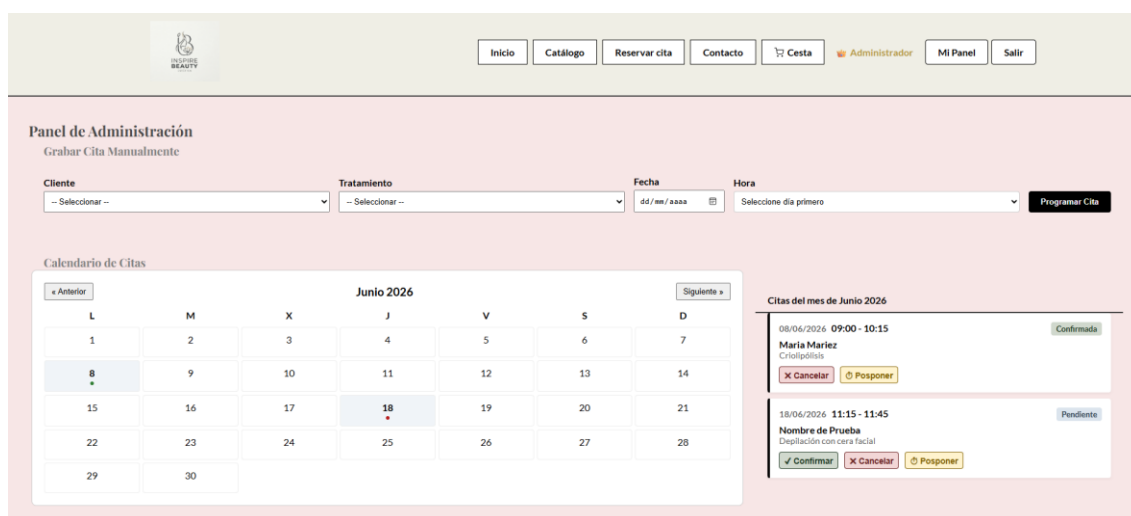
11.2. Gestión Avanzada del Calendario de Citas

El corazón organizativo de la clínica reside en el módulo de gestión de agenda (/admin/reservar). Para facilitar la labor de recepción, se ha diseñado una interfaz de pantalla dividida (*Split Screen*) que combina un calendario visual interactivo con un panel de control operativo.

1. Interfaz Interactiva de Calendario: Se ha programado desde cero, utilizando *Vanilla JS*, un calendario de vista mensual. Al cargar la página, el navegador recibe los datos de la base de datos y dibuja un indicador visual (puntitos de colores) en cada día que contiene citas. Los colores actúan como un código rápido de estado: rojo para citas pendientes de revisar y verde para citas ya confirmadas. Al hacer clic sobre cualquier día del calendario, la cuadrícula filtra instantáneamente el listado lateral, mostrando únicamente la agenda detallada de la jornada seleccionada, ordenando los pacientes de manera cronológica.

2. Operativa sobre Citas (Confirmar, Cancelar y Posponer): Desde el listado diario, el personal de clínica tiene un control total sobre el estado de las citas que entran por la plataforma web. Cada solicitud permite tres acciones transaccionales directas:

- **Confirmar / Cancelar:** Modifica el estado de la cita en la base de datos.
- **Posponer (Reschedule):** Si existe un contratiempo en la clínica, el administrador puede desplegar un formulario interno en la propia tarjeta de la cita, seleccionar una nueva fecha y hora, y guardarlo sin necesidad de borrar la cita original y crear una nueva. Cada una de estas acciones dispara automáticamente el **Simulador de Correos**, enviando una notificación asíncrona al cliente para avisarle del cambio de estado de su sesión.



Panel de Administración
Grabar Cita Manualmente

Cliente:
 Tratamiento:
 Fecha:
 Hora:

Calendario de Citas

Junio 2026

L	M	X	J	V	S	D
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

Citas del mes de Junio 2026

08/06/2026 09:00 - 10:15 Confirmada
 María Maríez
 Cridolpólis

18/06/2026 11:15 - 11:45 Pendiente
 Nombre de Prueba
 Depilación con cera facial

Ilustración 15: Vista del panel Administrador – Calendario interactivo con citas pendientes y confirmadas

3. Agendado Manual (Atención en Mostrador o Teléfono): La plataforma no solo atiende a clientes digitales, sino que actúa como el software TPV (*Terminal Punto de Venta*) principal de la clínica. Si un cliente llama por teléfono o acude al mostrador, el administrador dispone de un formulario de inserción manual.

- **Asistencia AJAX en Tiempo Real:** Fiel al compromiso con la Experiencia de Usuario (UX) del *staff*, este formulario se comunica con el servidor mediante peticiones asíncronas (fetch / AJAX). Cuando el administrador selecciona un tratamiento y una fecha en el mostrador, el desplegable de la hora se alimenta dinámicamente consultando las citas ya aprobadas, mostrando única y exclusivamente los huecos reales en los que cabe la duración del tratamiento deseado, previniendo así errores humanos o solapamientos.

11.3. CRM y Gestión de Clientes (Modo Mostrador)

Una plataforma de gestión clínica no estaría completa sin considerar la realidad omnicanal del negocio: no todos los clientes provienen de la vía digital. Existe un porcentaje de usuarios (transeúntes locales o personas mayores) que acuden presencialmente al mostrador o interactúan exclusivamente por vía telefónica.

Para integrar a estos usuarios en el ecosistema digital sin generar duplicidades, el módulo de administración incorpora un Gestor de Clientes (Mini-CRM) y una lógica específica conocida internamente como "Modo Mostrador":

1. Base de Datos Centralizada (CRM): El administrador cuenta con una vista dedicada (/admin/clientes) donde puede auditar la totalidad de la base de datos de usuarios. Esta vista permite consultar el volumen de altas y obtener los datos de contacto (Email y Teléfono) de cualquier paciente registrado en caso de tener que realizar comunicaciones administrativas o campañas de fidelización.

2. Creación Rápida de Clientes Físicos: Durante el proceso de Agendado Manual de citas (visto en el apartado anterior), el administrador no está obligado a seleccionar un cliente ya existente. Si la persona que llama es nueva, la plataforma permite registrarla "al vuelo"

pulsando la opción "+ Nuevo Cliente". Esto despliega un mini-formulario que captura únicamente el Nombre, Email y Teléfono, permitiendo cerrar la cita en segundos.

3. Lógica Backend (*Shadow Accounts*): El reto técnico de registrar a un cliente desde el mostrador radica en que el administrador no debe conocer ni asignar contraseñas privadas. Para solucionar esta brecha, se diseñó un sistema de "Cuentas en la Sombra" (*Shadow Accounts*). Cuando el Backend detecta una creación desde el mostrador, genera automáticamente una contraseña técnica de seguridad (la constante CUENTA_MOSTRADOR), la cual es encriptada inmediatamente en la base de datos mediante *Bcrypt*.

Esta solución cumple un doble propósito:

- Satisface la integridad referencial de la base de datos (toda cita necesita obligatoriamente pertenecer a un user_id válido).
- Deja la cuenta en un estado latente para su futura adopción digital. Si ese cliente físico decide usar la página web y trata de registrarse o iniciar sesión, el sistema detecta la bandera CUENTA_MOSTRADOR. En lugar de darle un simple error, **la plataforma le guía proactivamente hacia un flujo específico de "Completar Registro"**, donde el cliente puede establecer su propia contraseña privada y tomar el control total de su historial.

Adicionalmente, el sistema se ha robustecido implementando también un flujo estándar de **"Recuperar Contraseña"**, garantizando que ningún usuario se quede bloqueado fuera de la plataforma en ninguna circunstancia.

11.4. Control Comercial y Catálogo de Tratamientos

El último pilar del módulo de administración es el Gestor de Tratamientos (/admin/tratamientos). El objetivo técnico de esta sección es otorgar total autonomía comercial al personal de la clínica, permitiéndoles modificar la oferta de la empresa sin requerir asistencia técnica ni conocimientos de gestión de bases de datos.

La interfaz presenta un listado tabular de todos los servicios almacenados en la base de datos (tanto faciales como corporales), y permite realizar tres operaciones fundamentales de negocio en tiempo real:

1. Gestión de Disponibilidad (*Toggle Status*): No todos los tratamientos están disponibles durante todo el año. Mediante un simple botón, el administrador puede cambiar el estado de un tratamiento entre *Activo* o *Inactivo*. Al desactivar un tratamiento, este desaparece instantáneamente de todas las vistas públicas del cliente (Catálogo general, Promociones y desplegables de Reservas de Cita), impidiendo su venta hasta que la clínica vuelva a habilitarlo.

2. Actualización Dinámica de Precios: La plataforma permite introducir modificaciones directas sobre el PVP (Precio de Venta al Público) de cada servicio. Esta funcionalidad incluye medidas de validación en el servidor (*Backend*) para asegurar que no se introduzcan valores negativos o formatos erróneos. Al guardar, el nuevo precio se actualiza globalmente en el sistema, afectando a la cesta de la compra y a los tickets de caja inmediatamente.

3. Control de Promociones (*Marketing*): Para potenciar campañas de marketing puntuales (como rebajas de temporada o el *Black Friday*), el administrador dispone de un selector que

marca un tratamiento como "En Promoción". Al activar este *flag* (bandera lógica en la base de datos), el sistema reorganiza la vista pública del cliente, insertando automáticamente el tratamiento en la pestaña especial de "Promociones" y destacándolo visualmente en el catálogo para incentivar la conversión comercial.

11.5. Gestión de Venta Física en Mostrador

El diseño de la plataforma asume un modelo de negocio omnicanal. Aunque la pasarela de pago virtual gestiona de forma autónoma a los clientes digitales, el módulo de administración facilita la operativa física del local.

Actualmente, el sistema opera bajo un **modelo de registro híbrido**. Cuando un cliente acude presencialmente para adquirir un bono o tratamiento, el administrador utiliza el panel interno para registrar la transacción en la base de datos (garantizando así que los indicadores financieros del *Dashboard* se actualicen correctamente). Sin embargo, la pasarela de cobro real se delega a los sistemas tradicionales del comercio (cobro en efectivo en caja o cobro mediante un terminal TPV bancario físico).

Nota: La integración nativa de un módulo TPV digital propio dentro del panel del administrador, que centralice contablemente ambos flujos, ha sido proyectada como una de las principales líneas futuras de desarrollo de la plataforma.

12. Plan de Pruebas

Para garantizar la calidad y robustez del software antes de su entrega definitiva, se ha diseñado e implementado un plan de pruebas funcional basado en la metodología de Caja Negra (Black Box Testing), enfocado estrictamente en la validación de las entradas y salidas de la aplicación bajo el comportamiento esperado por el usuario.

12.1. Pruebas de Interfaz, Navegación y Validación

En la capa de presentación se han ejecutado los siguientes casos de prueba críticos para asegurar una Experiencia de Usuario (UX) libre de fricciones:

- **Pruebas de Adaptabilidad (*Responsive Design*):** Se ha verificado el comportamiento de la cuadrícula (*Grid*) en múltiples resoluciones. Se confirma que, al descender de los 768px de ancho de pantalla (punto de quiebre o *Breakpoint* principal), los menús colapsan correctamente y los contenedores de los tratamientos pasan fluidamente de una distribución en filas a un apilamiento vertical, garantizando la legibilidad en *smartphones*.
- **Comportamiento de Estados Visuales:** Se ha testado la correcta renderización de la paleta de colores. En concreto, se ha verificado la legibilidad de la tipografía mediante el intercambio y contraste de colores (beige corporativo frente a rosa pastel) durante los eventos interactivos (:hover) del ratón en botones y menús desplegados.
- **Validación Proactiva de Formularios:** Se han realizado intentos de inyección de datos incompletos en los formularios de Registro, Contacto y *Checkout*. Se certifica que la lógica del lado del cliente detecta las anomalías (correos sin arroba, teléfonos con caracteres alfabéticos o campos vacíos) y detiene nativamente el flujo de envío de datos hacia el servidor, mostrando alertas tempranas al usuario.

- **Pruebas de Interactividad Asíncrona (AJAX):** Se ha probado el motor dinámico de selección de horas en las reservas. Se confirma que, al modificar el tratamiento o la fecha elegida, el navegador lanza exitosamente la petición asíncrona (*fetch*) y reconstruye el desplegable de horas disponibles sin provocar recargas forzosas ni pantallazos blancos en la interfaz.

ID	Caso de Prueba	Descripción	Resultado Esperado	Estado
T-01	Navegación	Clic en los enlaces del menú superior.	Redirección fluida entre páginas sin errores 404.	Pasado
T-02	Responsivo	Redimensionar navegador a menos de 768px.	Las rejillas se apilan y el menú adopta formato móvil.	Pasado
T-03	Feedback	Pasar el cursor (:hover) sobre botones y enlaces.	Cambio suave de color/escala mediante transición CSS.	Pasado
T-04	Vacíos	Enviar formularios sin rellenar datos obligatorios.	El navegador bloquea el envío de forma nativa.	Pasado



T-05	Formato Email	Introducir correo sin "@" o dominio válido.	El navegador o script detiene el flujo pidiendo formato válido.	Pasado
T-06	Cita Previa	Enviar formulario de reservas conforme.	Validación en cliente y simulación de éxito visual.	Pasado
T-07	Contraseñas	Introducir claves diferentes en registro.	El sistema lanza un aviso controlado y detiene el alta.	Pasado
T-08	Botón Scroll	Pulsar la flecha flotante al final de la página.	Desplazamiento suave (scroll-behavior: smooth) al inicio.	Pasado

Tras la ejecución de la batería de tests, se certifica que el comportamiento visual y la lógica del Frontend responden de manera conforme. Los flujos de validación detienen los envíos erróneos —como se ha evidenciado en las ilustraciones de apartados anteriores— garantizando una experiencia de usuario (UX) óptima.

12.2. Entorno Tecnológico y Herramientas de Testing

Para la validación del comportamiento de la interfaz y la depuración del código, se emplearon de forma exhaustiva las herramientas de desarrollo integradas (DevTools) de los navegadores Google Chrome y Mozilla Firefox, garantizando así la compatibilidad cruzada (Cross-Browser).

Componente / Herramienta	Tipo	Propósito	Estado
Google Chrome / DevTools	Navegador	Inspección de elementos HTML y depuración de scripts.	Conforme
Mozilla Firefox	Navegador	Test de compatibilidad cruzada de reglas Grid y Flexbox.	Conforme
Safari (iOS Mobile)	Entorno Móvil	Test de usabilidad táctil y adaptabilidad responsiva.	Conforme
Nu HTML Checker (W3C)	Auditoría	Validación oficial de la semántica del marcado HTML5.	Conforme

12.2.1. Auditoría de Estándares Web con W3C

Como parte del compromiso con la calidad del software, el código fuente HTML resultante de las vistas generadas por PHP fue sometido a validadores de marcado internacionales (Nu HTML Checker del consorcio W3C). Durante este proceso se auditaron y corrigieron inconsistencias semánticas menores, como campos ocultos huérfanos y omisiones de atributos obligatorios (por ejemplo, el atributo action en algunos formularios), logrando certificar el código para que ofrezca una respuesta 100% libre de errores estructurales (ver Ilustración 13).



Nu Html Checker Usó de memoria: 179 MB

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for uploaded file registro.html (checked with vnu 26.5.21)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options...](#)

Check by [file upload](#) [Seleccionar archivo](#) Ningún archivo seleccionado

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check](#)

Use the Message Filtering button below to hide/show particular messages, and to see total counts of errors and warnings.

[Message Filtering](#)

- Error** Bad value for attribute `action` on element `form`: Must be non-empty.
From line 43, column 17; to line 43, column 72

```
<form action="" class="form-contacto" id="formRegistro">
```
- Info** To set the document's location as the action for a form, omit the action attribute.
From line 43, column 17; to line 43, column 72

```
<form action="" class="form-contacto" id="formRegistro">
```

Ilustración 16: Reporte inicial de la W3C detectando un error de atributo no válido en el formulario de registro.

Siguiendo las recomendaciones técnicas emitidas por el propio organismo validador, se procedió a la subsanación inmediata del código fuente en Visual Studio Code, omitiendo el atributo `action=""` por completo de la etiqueta `<form>`, delegando así la gestión completa del evento y el tratamiento de los datos en el script desarrollado en JavaScript nativo.

Tras aplicar la corrección y reevaluar el documento en el servidor de la W3C, se obtuvo el reporte definitivo de conformidad total (ver Ilustración 13), certificando que el Frontend está 100% libre de errores de marcado y cumple con las directrices oficiales de la Web Semántica.

Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for registro.html (checked with vnu 26.5.21)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options...](#)

Check by [file upload](#) [Seleccionar archivo](#) Ningún archivo seleccionado

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check](#)

Document checking completed. No errors or warnings to show.

Used the HTML parser.

Total execution time 5 milliseconds.

[About this checker](#) • [Report an issue](#)

Ilustración 17: Reporte final de conformidad de la W3C tras la corrección del marcado semántico en la interfaz.

12.3. Casos Reales de Depuración y Resolución (Troubleshooting)

Durante la fase de Testing, el equipo se apoyó en registros de seguimiento (Walkthroughs) para documentar los errores lógicos (Bugs) detectados y su resolución arquitectónica. Destacan dos intervenciones críticas:

- **1. Conflicto de Herencia CSS (La Flecha Interactiva):** Durante las pruebas visuales cruzadas, se detectó que el botón flotante de "Subir al inicio" cambiaba a colores inconsistentes dependiendo de la página visitada. Mediante las *DevTools*, se rastreó el problema hasta descubrir una colisión de estilos en los diferentes archivos (*estilos2.css*, *estilos3.css*). La resolución consistió en centralizar la clase *.flecha* en el archivo raíz, eliminando código redundante y aplicando el principio de desarrollo DRY (*Don't Repeat Yourself*).
- **2. Mejora de la Lógica de Estado en la Interfaz (El Contador de la Cesta):** Se observó que, cuando un usuario no tenía productos en el carrito, la cabecera mostraba de forma poco estética un texto "Cesta (0)". Se refactorizó la vista principal implementando una estructura condicional ternaria en PHP (`<?=$cartCount > 0 ? "($cartCount)" : "" ?>`) que ocultaba el contador dinámicamente si el valor era nulo. Además, se sustituyó el texto estático por un icono SVG nativo incrustado en línea, optimizando el peso y diseño de la barra de navegación.
- **3. Restricciones de Estilo en Clientes de Correo (Simulador EmailService):** Al testear la generación automática de los tickets electrónicos y confirmaciones, se descubrió que los estilos visuales de la plantilla no se renderizaban correctamente al intentar enlazar archivos *.css* externos (una restricción de seguridad habitual en gestores de correo reales como Gmail). Se solventó recodificando íntegramente la arquitectura de la plantilla a nivel de servidor, inyectando los atributos visuales mediante estilos en línea (*Inline CSS*) generados al vuelo por PHP, garantizando un renderizado consistente independientemente del cliente de correo.
- **4. Gestión Asíncrona de Disponibilidad de Citas:** Durante las pruebas de "Caja Negra" del motor de reservas, se descubrió que el formulario generaba visualmente horas que ya estaban ocupadas (aunque el servidor las bloqueaba exitosamente *a posteriori* por seguridad). Para eliminar esta fricción de la Experiencia de Usuario (UX), se refactorizó la arquitectura creando una pequeña API en el *Backend*. Esta mejora permitió al navegador consultar la disponibilidad al servidor de forma "silenciosa" mediante peticiones asíncronas (**AJAX con fetch**), mostrando al usuario única y exclusivamente las horas reales disponibles sin recargar la página.

13. Conclusiones

El desarrollo del proyecto "Inspire Beauty" ha supuesto un reto técnico y organizativo integral, logrando transformar de forma exitosa los procesos analógicos y tradicionales de un centro de estética en un ecosistema digital moderno, autónomo y transaccional.

13.1. Cumplimiento de Objetivos del Frontend y Experiencia de Usuario

Desde la perspectiva de la capa de presentación, la plataforma ha cumplido sobradamente su objetivo fundacional: establecer una identidad corporativa premium que transmita

profesionalidad y confianza al cliente desde el primer clic. La firme decisión arquitectónica de prescindir de frameworks pesados (como Bootstrap) en favor del código nativo (Vanilla CSS y Vanilla JS) ha demostrado ser un acierto rotundo. Se ha logrado una velocidad de carga máxima, una adaptabilidad perfecta a dispositivos móviles (Mobile First) y un control absoluto sobre cada píxel del diseño. El resultado es una interfaz intuitiva que guía al usuario por un embudo de conversión claro, facilitando el acceso a la información y culminando en una experiencia de compra y reserva de citas sin fricciones.

13.2. Cumplimiento de Objetivos del Backend y Lógica de Negocio

A nivel de servidor y arquitectura interna, el proyecto ha superado con creces los requerimientos básicos de una aplicación web tradicional. La decisión de desarrollar un **entorno MVC personalizado en PHP Nativo**, evitando *frameworks* de terceros, ha cristalizado en un sistema altamente modular, seguro y fácil de auditar. La gestión de la persistencia mediante consultas preparadas (PDO) sobre SQLite y el riguroso sistema de encriptación de credenciales (Bcrypt) garantizan la integridad de los datos de la clínica. Además, la implementación de soluciones ingeniosas como el Simulador Interno de Correos (*EmailService*), el sistema de Cuentas en la Sombra (*Shadow Accounts*) para el mostrador, y la integración asíncrona (AJAX) para el control cruzado de disponibilidades, demuestran una capacidad sobresaliente para resolver problemas de negocio reales mediante el código. En definitiva, se ha entregado una herramienta de gestión administrativa robusta, que centraliza y automatiza el 100% de la operativa del centro estético.

13.3. Coordinación y Trabajo en Equipo

Más allá de los logros puramente técnicos, el mayor éxito del proyecto ha sido la simulación de un entorno de trabajo profesional. La estricta separación de responsabilidades entre el diseño de la capa de presentación (Frontend) y el desarrollo de la lógica de servidor (Backend) ha obligado al equipo a establecer protocolos de comunicación rigurosos.

La adopción temprana de metodologías ágiles basadas en el flujo Kanban (mediante Trello) proporcionó una visión transparente del progreso diario, previniendo cuellos de botella. Paralelamente, el uso centralizado de Git y GitHub garantizó un control de versiones impecable, permitiendo la integración continua y la fusión de ambas capas de código sin riesgo de pérdida de datos. Esta sinergia demuestra la capacidad del equipo para escalar proyectos, adaptarse a las herramientas del sector y aplicar buenas prácticas corporativas en un entorno real.

13.4. Aprendizaje Personal y Futuras Mejoras

A nivel académico y profesional, el desarrollo de "Inspire Beauty" ha marcado un punto de inflexión en las capacidades técnicas de ambas desarrolladoras. La transición desde la creación de páginas web puramente estáticas hacia el dominio de una Arquitectura MVC real y escalable, respaldada por bases de datos relacionales y código nativo, ha consolidado de manera definitiva los conocimientos adquiridos durante todo el Grado Superior.

Como todo software vivo, la plataforma es un producto susceptible de evolución constante. Para dar continuidad al proyecto, se han definido las siguientes Líneas Futuras de Desarrollo:

Integración de Pasarelas Bancarias Reales: Sustitución de la pasarela de simulación por conexiones nativas a APIs del entorno financiero (Stripe, Redsys o pago mediante Bizum), habilitando el comercio electrónico 100% automatizado.

Módulo TPV Contable Interno: Creación de una vista de caja registradora virtual en el panel de administrador que permita procesar cobros en metálico de clientes físicos, unificando en una sola base de datos la contabilidad presencial y digital de la clínica.

Configuración de un Servidor SMTP: Migración del componente EmailService actual (simulador local) a integraciones con servidores de correo reales (como Mailtrap o SendGrid), automatizando el envío real de notificaciones y recordatorios a la bandeja de entrada personal de los clientes.

14. Bibliografía y Webgrafía

Para el correcto desarrollo, fundamentación teórica y resolución de problemas técnicos a lo largo del proyecto, se ha consultado la documentación oficial de las siguientes tecnologías y estándares:

World Wide Web Consortium (W3C). HTML5 Specification. Consultado para garantizar la correcta semántica de las etiquetas estructurales. Disponible en: <https://www.w3.org/TR/html5/>

Mozilla Developer Network (MDN Web Docs). CSS Grid, Flexbox y Fetch API. Referencia principal para la maquetación responsiva avanzada y la comunicación asíncrona (AJAX) del lado del cliente. Disponible en: <https://developer.mozilla.org/>

The PHP Group. PHP Official Documentation (PHP 8). Guía principal para la programación de la lógica de servidor orientada a objetos, uso de sesiones, conexiones seguras mediante PDO y cifrado de contraseñas (Bcrypt). Disponible en: <https://www.php.net/docs.php>

SQLite Consortium. SQLite Official Documentation. Referencia para el diseño del modelo entidad-relación y consultas estructurales en entornos serverless. Disponible en: <https://www.sqlite.org/docs.html>

Google Fonts. Librería Tipográfica. Implementación e importación de las familias tipográficas Playfair Display y Lato para la conformación de la identidad visual. Disponible en: <https://fonts.google.com/>

Pexels, Unsplash y Google Imágenes: Bancos multimedia con licencias libres utilizados con fines estrictamente académicos y educativos para el prototipado visual de la interfaz de usuario. Disponibles en: <https://www.pexels.com>

SweetAlert2. JavaScript Popup Boxes. Librería utilizada puntualmente para la sustitución de las alertas nativas de JavaScript por ventanas modales responsivas y accesibles. Disponible en: <https://sweetalert2.github.io/>

Trello & GitHub. Documentación Oficial. Consultas de flujos de trabajo sobre metodologías ágiles (Kanban) y resolución de conflictos de ramas (merges) en control de versiones.

Anexos

Anexo I: Wireframes del Proyecto y Comparativa de Interfaz

Con el objetivo de optimizar la extensión de la memoria, mantener el límite de páginas establecido y evitar la dispersión de hojas, se presentan de forma agrupada los bocetos iniciales de baja fidelidad (*wireframes*) frente a su maquetación final renderizada en el navegador web.

1. Catálogos de Servicios (Facial y Corporal)

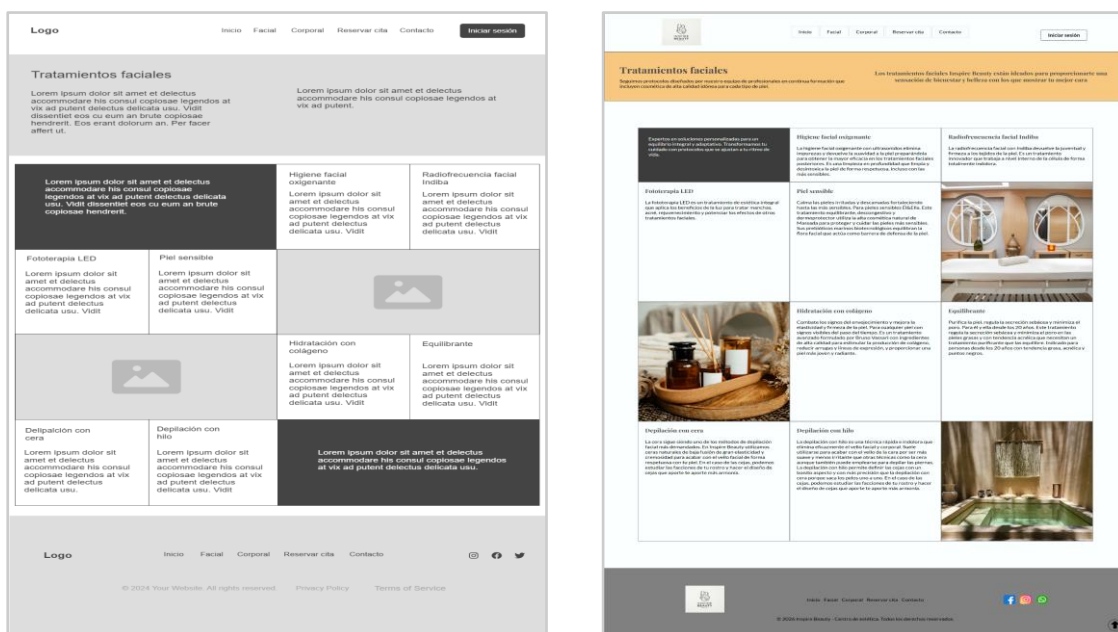


Figura A.1: Estructura visual de los catálogos de servicios. Distribución de los tratamientos estéticos de la marca mediante rejillas flexibles (CSS Grid) y optimización de imágenes multimedia con feedback interactivo.

2. Módulo de Reservar Cita Online

The figure displays two wireframe designs for the online booking module. Both designs feature a symmetrical layout with a header, a main content area, and a footer. The left design includes a 'RESERVA TU MOMENTO' section on the left and a 'SOLICITA TU CITA' form on the right. The right design includes a 'RESERVA TU MOMENTO' section on the left and a 'SOLICITA TU CITA' form on the right, with a 'Toma tu cita' button. The forms include fields for name, email, phone, treatment selection, and date. The footer contains copyright information and links to privacy policy and terms of service.

Figura A.2: Arquitectura del módulo de conversión para reservas online. Disposición simétrica de las pautas informativas del centro estético frente al formulario controlado con campos obligatorios.

3. Control de Acceso (Login y Registro)

The figure displays two wireframe designs for the login and registration module. Both designs feature a symmetrical layout with a header, a main content area, and a footer. The left design includes a 'INSPIRE BEAUTY' header, a 'INICIA SESIÓN' form on the left, and a large image placeholder on the right. The right design includes a 'INSPIRE BEAUTY' header, a 'INICIA SESIÓN O REGÍSTRATE' form on the left, and a large image placeholder on the right. The forms include fields for email, password, and a 'REGÍSTRATE' button. The footer contains copyright information and links to privacy policy and terms of service.

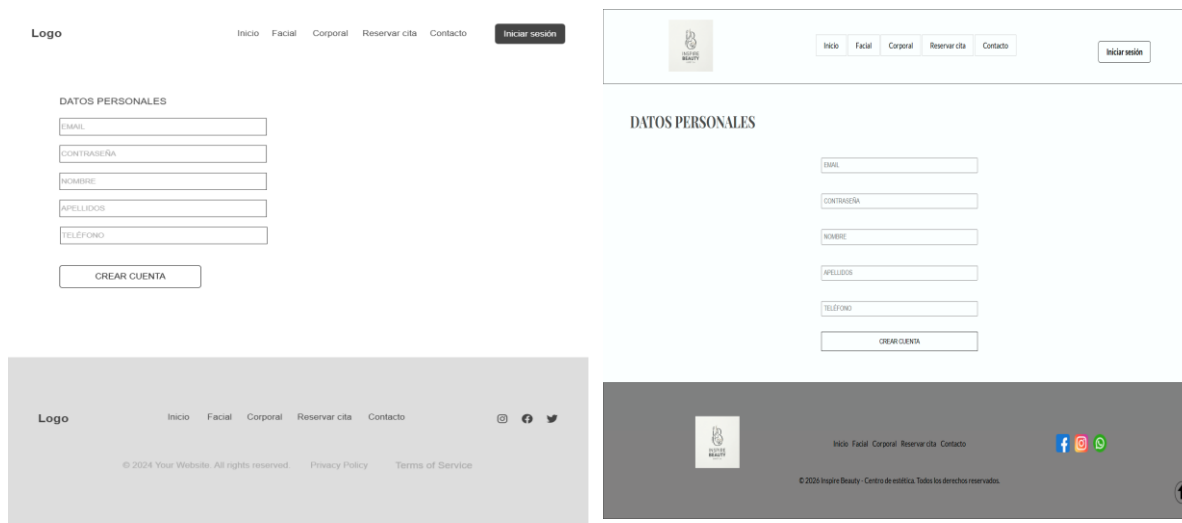


Figura A.3: Pantallas del módulo de control de acceso. Diseño unificado del formulario de inicio de sesión con pasarela de enlace directo hacia la interfaz de registro de datos personales.

Anexo II: Guía de Estilo Detallada

- **Tipografías:** Estructura jerárquica basada en *Playfair Display* (Encabezados h1, h2, h3 con font-weight: 700) y *Lato* (Cuerpo de texto y menús).
- **Colores:** Paleta cromática corporativa fundamentada en grises oscuros suaves para mitigar la fatiga visual, combinada con fondos en blanco puro y tonos pastel empolvados como el rgb(247, 230, 230).
- **Interactividad CSS:** Estilización de los estados `:hover` mediante transiciones coordinadas (`transition: 0.3s`) y transformaciones de escala (`transform: scale(1.1)`) instaladas en las clases de botones de acción (`.btn-sesion`, `.btn-registro`, `.btn-cuenta`) para ofrecer una respuesta visual fluida al usuario.